

Binary Two-Way Automata and Corank Budgets in Brauer Monoids

Sharp compression, exponential amplification,
and exact finite verification

Prepared with ChatGPT for Vladimir Reshetnikov
7 October 2026

Main quantitative result. For every $n \geq 14$, an n -state one-way nondeterministic automaton over the fixed alphabet $\{0, 1\}$ has no equivalent two-way deterministic automaton with fewer than

$$\frac{1}{4} \left(\frac{5}{2} \right)^{\lfloor (n-14)/54 \rfloor} - \frac{1}{4}$$

states, under the explicitly specified endmarker model.

Abstract

We develop and sharpen the matching-monoid method in the September 2026 OpenAI manuscript on one-way liveness, in the context of recent automata reports in ProveIt. The central algebraic observation is a sharp, repetition-independent budget: the corank of any word in finitely many Brauer idempotents is at most the sum of their individual coranks. A short proof follows from the classical colored-component description of idempotents. This budget controls both nested corners in a relation-addition amplifier, giving the degree lower bound $2(5/2)^{\lfloor (h-2)/9 \rfloor}$ for a Brauer monoid having the full relation monoid on h points as a divisor.

On the automata side, we construct a symbol-local matching representation of a binary s -state deterministic two-way automaton in degree at most $8s + 2$. A four-letter compiler on $2h$ relation states, followed by two-bit decoding, gives a completely defined binary one-way nondeterministic language with $6h + 2$ states and the required relation divisor. Together these yield an explicit exponential binary determinization lower bound with coefficient $\log_2(5/2)/54$ in the exponent. We also prove a sharp support bound with equality, enumeration, and stability, and give an optimized finite recurrence. A separate binary complementation consequence is stated with its external theorem dependency identified.

All arguments for the determinization theorem are given here. Exact finite audits accompany the proofs; they are diagnostic evidence, not a replacement for proof or a Lean formalization. Attribution, source versions, limitations of novelty claims, and a focused research agenda are included.

[write] **Status and trust boundary, added 7 October 2026.** This research article was delivered to ProveIt’s docs/incoming intake (non-Gowers intake NG3) and is filed as the single-source report **binary-two-way-corank** in the automata category of the research-report collection (provenance, the sources as the write read them, what was checked, relation to the repository, the collected non-claims and the reading conventions are in Section 1.5). Its title page and PDF author field read “Prepared with ChatGPT for Vladimir Reshetnikov”, and its review of its own proofs “was an additional AI-assisted mathematical review, not independent human peer review” (Section 6). It is unrefereed and not formalized. *What is proved in the text:* Theorem 1.1 (the binary determinization bound), with Lemmas 2.1, 2.7, 2.8, 2.9, Theorems 2.2, 2.5, 2.12 and 2.13, Corollaries 2.3 and 2.6, Propositions 2.11, 2.14 and 2.15, Theorems 3.1 and 3.4, Theorem 4.1, Corollary 4.2, Proposition 4.3, Lemma 4.4, and Propositions 7.1 and 7.2. *What is conditional:* Theorem 5.4 (binary nondeterministic complementation) assumes Theorem 5.1, a theorem of a companion OpenAI manuscript that is stated here but not proved and was not checked by the write; it is *not* a proved result of this report, and neither is anything that cites it. Lemma 5.2 and Proposition 5.3 are proved here, the former as a normalization of the companion’s construction. *What is prior:* the cap-component classification (Dolinka et al.), the relation-corner and tree-contour architecture (the OpenAI liveness manuscript, adapted with attribution), and fixed-alphabet transfer (TheoremDB R816). No statement, proof or number of the source was changed.

Contents

1	Problem, source review, and contribution	1
1.1	What was reviewed	1
1.2	The machine convention	2
1.3	Three ingredients	2
1.4	Attribution and the status of the improvements	3
1.5	Provenance, sources and reading conventions	3
2	Corank budgets and relation-monoid divisors	6
2.1	Matching diagrams and their cap components	6
2.2	A budget for arbitrarily long words	8
2.3	Sharp support, equality, and stability	9
2.4	Corner reductions and unit lifts	10
2.5	Amplifying the rank cost of one relation addition	11
2.6	Choice of parameters and finite integer bounds	13
3	A linear-degree local representation of deterministic machines	15
3.1	Machine convention and statement	15
3.2	Acceptance becomes reachability of one sink	15
3.3	A sparse set of boundary slots	16
3.4	Contour wires and the two marker tests	17
3.5	Local contraction and correctness	18
3.6	Why the linear order cannot be improved universally	18
4	A two-register compiler and the binary lower bound	19
4.1	Four letters on two cyclic registers	19
4.2	A complete binary language with $6h + 2$ states	21
4.3	Singleton contexts force the full relation divisor	22
4.4	Proof and quantitative comparison	22
5	A separate consequence for binary complementation	23
5.1	The external path-diagram bound	23
5.2	Encoded contexts give the required antitone quotient	24

6	Exact finite audits and reproducibility	26
6.1	Idempotent structure	26
6.2	The repetition-independent budget	26
6.3	The relation compiler	27
6.4	Actual symbol-local matching diagrams	27
6.5	Exact recurrence and finite consequences	27
6.6	Reproduction and interpretation	28
7	Further research: precise questions and useful obstructions	28
7.1	The true degree of a full relation divisor	28
7.2	Beyond rectangular relation addition	29
7.3	Minimum generated rank needs more than union connectivity	29
7.4	Optimal compiler size and ambiguity	30
7.5	The constant in sparse matching representations	30
7.6	Ordered path monoids and nondeterministic complement	31
7.7	Uniformity and short witnesses	31
7.8	A practical formalization program	31
7.9	Research priorities	31
	References	33

1 Problem, source review, and contribution

The state cost of replacing nondeterminism by deterministic two-way head motion is a central succinctness problem for finite automata. Two-way and one-way finite automata recognize the same regular languages, but their state complexities can be very different. Sakoda and Sipser formulated the relevant simulation questions in 1978 [16]; Pighizzini’s survey gives the historical landscape [11]. A two-way target can revisit the input without keeping every previously read symbol in its finite control. A lower bound against unrestricted targets must therefore survive arbitrary interior reversals and arbitrarily long computations.

This article concentrates on one concrete research direction suggested by the two repositories: *how much of the recent full-relation lower-bound mechanism survives over a fixed binary alphabet, and how tightly can its two algebraic bottlenecks be controlled?* We give explicit answers, including all constants and the complete machine model.

1.1 What was reviewed

The repository review is a targeted mathematical review, rather than an endorsement of every manuscript in either collection. We pinned the inspected OpenAI release to commit `adc7f1241b42` and the inspected ProveIt tree to `670a67d35eda`. The accompanying source audit records the full commit identifiers, exact paths, and blob hashes. All statements about these repositories refer to these snapshots, accessed on 7 October 2026.

The principal OpenAI source is *An exponential two-way deterministic state lower bound for one-way liveness*, dated 25 September 2026 [9]. It supplies a matching-diagram approach to unrestricted two-way determinization. For full relations on an h -element set, its algebraic degree bound is

$$k \geq 2^{\lfloor (h-2)/31 \rfloor},$$

and its general machine representation uses at most $4(s+1)^2$ ports when an accepting initial configuration counts as success. The manuscript expressly works over an alphabet containing all 2^{h^2} binary relations. Its companion *An exponential state lower bound for two-way nondeterministic complementation* [8] uses an ordered path monoid and gives a different rank-amplification theorem. We inspect the relevant arguments and distinguish which ones are reproved here from which one is imported.

The formalization summary in the same OpenAI repository is a useful scope check [10]. It describes the formal results associated with the source theorem. It is not a formal certificate for the new estimates, compiler, or sparse representation in this article. We have neither extended nor rerun that Lean development.

Three nearby ProveIt report series make the choice of topic especially natural. The *shuffle-six-state-bound* report distinguishes reachability over a full transformation-pair alphabet from distinguishability over a fixed three-letter alphabet [15]. This distinction is directly relevant: a large-alphabet witness does not by itself settle the quantitative cost of a fixed-alphabet witness. The *DFAO reversal coloring obstruction* reports study sharp output-reversal and coloring phenomena [14]; the *constrained crossover closure* reports study finite fragments, closures, and binary NFA rank parameters [13]. These are neighboring questions with different machine models and invariants. They motivate a carefully scoped automata report, but no theorem from those three series is used as a premise below.

1.2 The machine convention

An input is $\vdash w \dashv$, where $w \in \Sigma^*$ and the two endmarkers are distinct and lie outside Σ . The head starts on $\vdash$ in one designated initial state. A transition reads the current symbol, changes state, and moves by -1 , 0 , or $+1$. It may never cross either endmarker. Deterministic transitions are partial functions; nondeterministic transitions are relations. Acceptance means that some finite run reaches a designated accepting state, including an initially accepting configuration. A nonaccepting infinite run rejects. All states, including initial and accepting states, are counted.

A *1NFA* has no left moves; the constructions here use right moves on ordinary symbols and only a final stay move to enter the accepting state on $\dashv$. A *2DFA* may move in either direction and stay. Thus the lower bound includes partial transitions, rejecting loops, and unrestricted reversals. Changing to a convention that requires a positive transition before initial acceptance can be handled with one extra initial copy in the target normalization; the resulting binary port bound is $8s + 10$. We state the sharper constant under the convention just fixed.

Theorem 1.1 (Binary determinization). *For every integer $h \geq 2$, there is an explicitly defined language $B_h \subseteq \{0, 1\}^*$ recognized by a $6h + 2$ -state 1NFA such that every s -state 2DFA recognizing B_h satisfies*

$$8s + 2 \geq 2 \left(\frac{5}{2} \right)^{\lfloor (h-2)/9 \rfloor}. \quad (1)$$

Consequently, for every $n \geq 14$ there is a language recognized by an n -state binary 1NFA for which every equivalent 2DFA satisfies

$$s \geq \frac{1}{4} \left(\frac{5}{2} \right)^{\lfloor (n-14)/54 \rfloor} - \frac{1}{4} = 2^{(\log_2(5/2)/54)n - O(1)}. \quad (2)$$

The exponent coefficient is $\log_2(5/2)/54 = 0.0244801499\dots$

The proof is assembled in Section 4.4. The familiar subset construction gives an exponential one-way deterministic upper bound and hence a two-way upper bound; with the present endmarker convention the harmless normalization costs only a constant number of states. Accordingly the worst-case binary 1NFA-to-2DFA state growth is $2^{\Theta(n)}$ under this model. The precise coefficient in the exponent is not claimed optimal. For small n , (2) may give only a weak numerical bound; the parity-aware recurrence in Section 2.6 is appreciably stronger for finite parameters.

1.3 Three ingredients

The proof connects three independently useful constructions.

First, for idempotents $e_1, \dots, e_v$ in the Brauer monoid $\mathcal{B}_m$, every word w in these generators satisfies

$$m - \text{rank}(w) \leq \sum_{i=1}^v (m - \text{rank}(e_i)).$$

Repetition of a generator costs nothing further in this bound. The proof uses the parity and invariance of the union of the generators' cap components. Applied twice to the source's relation-addition construction, it gives

$$k \geq F(h) := 2(5/2)^{\lfloor (h-2)/9 \rfloor}$$

whenever a submonoid of $\mathcal{B}_k$ maps onto the full relation monoid on h points. The choice of two arms of size five is optimal among fixed rectangular choices in this particular amplifier.

Second, symbol-local crossing slots represent an s -state 2DFA over an a -letter alphabet by matching diagrams of degree at most $2(a+2)s+2$. The bound is linear in s for a fixed alphabet. The accepting configuration's predecessor component is a tree; doubling its edges into a contour turns reachability into a matching connection. Inactive candidate edges receive private leaves, ensuring that each symbol's diagram is determined entirely by that symbol and the fixed transition table.

Third, two cyclic state registers implement every h -point relation using four fixed relation letters on $2h$ states. A two-bit decoder uses $6h$ ordinary states; adding one initial and one accepting state gives $6h+2$. Singleton contexts ensure that any monoid recognizing the complete binary language retains every relation product on the submonoid generated by canonical encodings. No homomorphism on all arbitrary binary strings is assumed.

1.4 Attribution and the status of the improvements

The cap-component classification is classical [4]. The sharp support bound and its equality and stability formulas are explicit consequences of that structure, proved here for their use and independent interest. The relation-corner architecture and deterministic tree-contour method are adaptations of the OpenAI liveness manuscript [9], with earlier contour ideas in reversible and halting simulations [17, 7]. The sharp corank budget changes the accounting, rather than merely optimizing a numerical parameter in the old support inequality.

Alphabet substitution itself is established methodology, including the full-automata approach of Yan [19]. Kapoutsis discusses binary liveness encodings in the few-reversal setting [6, Section 8]. More directly, TheoremDB record R816 contains an earlier explicit four-letter, $3h$ -state relation encoding and a $9h$ -state binary decoder, with a constant-state-factor macro transfer [18]. Its stated lower bound imports the quadratic liveness theorem of Adeogun and Kapoutsis [1]. The same transfer principle can be combined with a stronger large-alphabet theorem. We therefore do not claim that the mere possibility of a binary transfer originates here.

The specific contributions developed in this article are the sharp word corank budget, its two uses in the stronger amplifier, the sparse local matching degree bound, the smaller two-register compiler, and their explicit combination. We supply proofs and independent finite diagnostics, and make the source dependence reviewable. A targeted literature and repository review cannot certify priority against all existing work. These results are offered as a research manuscript for independent review, not as an announcement of established priority.

Finally, the state lower bounds do not establish $L \neq NL$ or any other uniform space separation. Uniform simulation, constructibility, and bounds tied to input length are separate issues. No such conclusion is used or asserted.

1.5 Provenance, sources and reading conventions

[write] **Added 7 October 2026.** *Provenance.* The archive `ProveIt_Binary_Two_Way_Corank.zip` (581,213 bytes, SHA-256 `e1817c4d2f42...65b36439c`; 32 files in one wrapper directory `Binary_Automata_Corank/`, 794,441 bytes unpacked) arrived in commit `964621dd2` (7 October 2026, 14:02) and was placed by commit `bdcae034b` as this new report, unprefixing, in its delivered layout: the main file `article.tex` with its seven section files (2,214 lines, 96,365 bytes), `references.bib`, `Makefile`, the programs under `code/`, the reference outputs under `certificates/`, `SOURCE_AUDIT.txt`, `source_manifest.json`, `THEOREM_LEDGER.csv`, `THIRD_PARTY_NOTICES.txt`, `LICENSE-APACHE-2.0.txt` and the delivery `README.txt` as `README.md` (29 files, byte-identical to the delivery). Not shipped, and retrievable from the arrival commit: the delivered 31-page `article.pdf`, the prebuilt `article.bbl` and the checksum manifest `SHA256SUMS` (31 entries, verified at placement). The delivery `README` was replaced by the report `README`. The text is dated 7 October 2026. Its author lines are recorded in the status note above; it names no human author.

It pins ProveIt commit 670a67d35eda2fc92f2483b1b3d643bad9173113 (7 October 2026, 13:09 Pacific time), which exists, and commit adc7f1241b42e322a6451854ab7e4b4c146bf78a of the external repository openai/math (SOURCE_AUDIT.txt). It is a single-source report, so no merge choice was made. The write prefixed the 91 delivered labels with `btc:` (so `alg:budget` is `btc:alg:budget`) and updated the 85 references to them (53 `\ref`, 32 `\eqref`; `\cref` is loaded but unused); THEOREM_LEDGER.csv keeps the delivered names. It added this subsection, the status note and the notes marked [write], each after the last delivered text of its section or after the title page, and no numbered statement, display or table, so every section, statement, equation, table and figure keeps its delivered number (aux files of builds of the delivered and the written text compared). The 20 numbers that the ledger quotes are the delivered numbers. No other text was changed.

The sources, as the write read them (7 October 2026).

- The repository at the pin: the three automata README blobs that the introduction and SOURCE_AUDIT.txt cite (`shuffle-six-state-bound` 077d0777, `dfao-reversal-coloring-obstruction` 92fa1fc9, `constrained-crossover-closure` e8ff27d7) are unchanged at this write, and the descriptions of them agree with them (checked at placement).
- The companion OpenAI manuscript, through a copy of `sections/diagrams.tex` at commit adc7f1241 fetched at the write (not a local checkout): its theorem “Order-reversing image bound”, labelled `amp:image` (and `amp:relation-bound`), has the hypotheses and the bound $2m \geq 2^{\lfloor (h-2)/127 \rfloor}$ printed in Theorem 5.1, including that the map need not preserve identities. Its proof, in `sections/amplification.tex`, was not read.
- Not read by the write: the OpenAI liveness manuscript, its Lean summary `lean/docs/129.md`, TheoremDB R816, and the classical and recent literature (Dolinka et al., East-Gray, Kapoutsis, Yan, Adeogun-Kapoutsis, Pighizzini and the rest); the manuscript’s account of them is reported, not confirmed. `source_manifest.json` hashes the producer’s local snapshots, none of which is shipped, so its hashes cannot be checked here.
- The bibliography file has 27 entries, of which 19 are cited and printed; the eight others (`CompanionComplementation2026`, `RabinScott1959`, `Shepherdson1959`, `Sipser1980`, `AdeogunKapoutsisLimitation2026`, `Chrobak1986Unary`, `Chrobak2003Errata`, `Kapoutsis2018ShortLiveness`) are never cited and so not printed. `CompanionComplementation2026` duplicates the cited `OpenAIComplement2026` with an unpinned `blob/main` link; the printed entry is the pinned one.

What was checked at intake. The placement dossier read the manuscript and found no error; it re-derived by hand the main arithmetic, the corank budget, the amplifier, the optimization, the compiler path analysis, the singleton-context quotients, Theorem 3.4 and the bounds of Proposition 7.1; with its own code it recomputed the idempotent counts for $m \leq 6$, the support equality counts, the budget on 30,756 random word prefixes, the union obstruction, $D(h)$ for $h \leq 256$, the optimal $p = q = 5$, the floor identities and the compiler’s exact path counts; and it reran the shipped suite on a copy (pass; every certificate equal as data except timings, the Python version and one output path). The sparse-slot construction and the imported premise were read, not re-derived. *What the write checked.* Every proof of Sections 2–4 and of Lemma 5.2, Proposition 5.3 and Theorem 5.4 (as a deduction from its premise) was read line by line, with no error found; the steps rechecked by hand are listed in the notes at the ends of Sections 2–5. With its own code (Python 3.14.4, `mpmath` 1.3.0), independent of the shipped programs: all perfect matchings for $m \leq 7$ (146,600), with idempotent counts 1, 1, 2, 10, 40, 296, 1936, 17872 and support equality counts 1, 1, 1, 7, 25, 61, 481, 2731 (equal to $\sum_q m!/(q!(m-3q)!)$), as tabulated in Section 6; the budget on every ordered idempotent pair and every two-generator monoid for $m \leq 4$ (the shipped counts 1,706, 1,898, 826 and 4,280 reproduced) and on 42,000 random word prefixes for $4 \leq m \leq 10$, with no violation; the union obstruction of Proposition 7.2; $D(h)$ for $h \leq 256$ against all 255 rows of `certificates/finite_bounds.csv` and the table of Section 6; the rates, the optimal (5, 5) over $p, q \leq 300$ and the floor identities; and the case counts quoted in Section 6. It reran the shipped suite on a copy (note at the end of Section 6). Finite computations are observations, not certificates.

Relation to the repository. Before this report no file of the repository treated two-way automata, Brauer monoids or relation-monoid divisors, and no Lean or Rocq declaration concerns them; no statement of this report is formalized, and its place in the collection confers no formal status. The three neighbouring automata reports it cites are context only: no theorem of theirs is a premise, and none shares a result with this report,

so no reciprocal note is proposed. The word “liveness” here names the one-way liveness languages of two-way automata theory; the “liveness beyond halting” of the Hilbert-tenth-problem reports is temporal liveness of computations, a different notion. The repository’s `lib/openai-math` holds an unrelated family of the external release; the manuscripts this report adapts are not vendored, and only the complementation section depends on one of them.

[write] What is not claimed, collected from the source. No priority certification: the targeted literature and repository review “cannot certify priority against all existing work”, the results are “offered as a research manuscript for independent review”, the corank budget’s priority is “unconfirmed” (ledger), and the negative search in East–Gray “is not a proof of novelty” (`SOURCE_AUDIT.txt`); the possibility of a binary transfer is not claimed to originate here (TheoremDB R816), and the exponential order of $\beta(h)$ already follows from the source. The coefficient $\log_2(5/2)/54$ is not claimed optimal, the choice $p = q = 5$ is optimal only within the rectangular amplifier, the constant 8 in $8s + 2$ is not claimed optimal, and the counting argument for block length does not bound compiler states. The machine model is fixed (two endmarkers, partial rules, acceptance at any time, all states counted); under a convention requiring a move before initial acceptance the port bound is $8s + 10$. No $L \neq NL$ or other uniform space separation, and nothing about running time, reversals or witness length. The finite audits are diagnostics, not proofs; nothing is formalized, the upstream Lean development was neither extended nor rerun, and no human peer review took place. Theorem 5.4 rests on an external premise that is not reproved. The write adds: its checks are exact finite computations or hand computations; it read the imported premise’s statement only.

Reading conventions (letters the source reuses, with the tempting false readings; no symbol of the source was renamed).

Symbol	Meaning here (where)	Not to be read as
s	the number of states of a 2DFA or 2NFA; in Proposition 2.15 and (21) the arm sum $p + q$	each other
k	the Brauer degree $\mathcal{B}_k$; in Proposition 2.11 the size $h - p - q + 1$	each other
m	the degree of $\mathcal{B}_m$; in Section 5 the number of labels per direction of $\mathcal{T}_m$	each other
a, b, t, r	the alphabet size a ; the four relation letters (Section 4); an idempotent a (Definition 2.10) and the idempotents a, b of Proposition 7.2; positive labels a, b (Lemma 5.2); t also the sink configuration (Section 3) and a word length; r also a rank, and $r(w)$ a relation product (Section 5)	each other
U, V	arm point sets (Proposition 2.11); register state sets (Section 4)	each other
F, E, P, Q	$F(h)$ the degree bound (18); $F \subseteq H$ a retained set; F, B, L, R the four relations of a path diagram; $E = I_F$; P, P_ℓ relations; $P \subseteq \mathcal{B}_m$ an accepting set; $Q = PEP$; Q a state set	each other; E_C an edge count
β, ρ, δ	$\beta(h)$ the least divisor degree (Section 7); $\beta = R_b(b_\ell)$ in the proof of Proposition 2.11; ρ a recognizer (Theorem 3.4) and a marker diagram (Lemma 5.2); δ the transition function and the deficit $\delta(e)$	each other
R, R_e	a relation; the corner map R_e (Lemma 2.7); TheoremDB record R816	each other
$D(h), D^\pm$	the recurrence (21); the label sets of $\mathcal{T}_m$	each other
L, J, C	an addition bound L , the language $L(C)$, a path relation L ; J the number of crossing candidates; C the candidate set, a union component, a 2NFA	each other
liveness	one-way liveness languages (two-way automata)	the “liveness beyond halting” of the Hilbert-tenth-problem reports

2 Corank budgets and relation-monoid divisors

The algebraic argument has two parts. First, the cap components of idempotent matching diagrams impose a corank budget that is independent of the length of a word in those idempotents. Second, this budget strengthens the relation-addition construction of [9]. We give the corner arguments and the complete induction, so the degree bound below does not depend on importing the source manuscript’s exponential bound as an unproved assumption.

2.1 Matching diagrams and their cap components

Write $[m] = \{1, \dots, m\}$, with $[0] = \emptyset$. The *Brauer monoid* $\mathcal{B}_m$ consists of the perfect matchings on two copies of $[m]$, called the left and right ports. A matching edge between opposite sides is a *transversal*; an edge whose endpoints are on the same side is a *cap*. Multiplication places the first factor to the left of the second, identifies equally indexed ports at the interface, and follows paths between the remaining outer ports. Components with no outer port are discarded. Parallel

edges at an interface are retained. This is the usual diagram multiplication associated with Brauer's algebras; see [3, 2, 5].

Associativity follows by contracting paths in a substring of a glued diagram: the contraction preserves exactly the connections available to the rest of the diagram. The identity I_m consists of the m fixed transversals. Set

$$\begin{aligned} \text{rank}(a) &= \#\{\text{transversals of } a\}, & \text{corank}(a) &= m - \text{rank}(a), \\ \text{supp}(a) &= \{i : \text{left } i \text{ is not paired with right } i\}. \end{aligned}$$

In particular, $\text{corank}(a)$ is even: each side has $\text{corank}(a)/2$ caps. Following disjoint transversal paths through a product gives

$$\text{rank}(ab) \leq \min\{\text{rank}(a), \text{rank}(b)\}. \quad (3)$$

The rank- m diagrams are precisely permutations, and these are exactly the units. A rank- m idempotent is therefore I_m .

The *cap graph* of a diagram has vertex set $[m]$: draw one red edge for each left cap and one blue edge for each right cap, identifying the two port labels but retaining the colors and parallel edges. Its degrees are at most two, with at most one incident edge of each color. A connected component is consequently an alternating path, an alternating cycle, or an isolated vertex. A two-vertex cycle means two parallel edges of different colors.

Lemma 2.1 (Idempotent component description). *For an idempotent $e \in \mathcal{B}_m$, every cap component is either an alternating path with an odd number of vertices, or an alternating cycle with an even number of vertices. Each path component carries exactly one transversal of e , joining its two missing-color endpoints; an isolated vertex carries a fixed transversal. A cycle component carries no transversal. In particular, every edge of e , including every transversal, stays within one cap component.*

Conversely, any collection of such colored components, completed by these transversals, defines an idempotent. A path on $2a + 1$ vertices contributes $2a$ to corank, and a cycle on $2b$ vertices contributes $2b$.

Proof. This is the classical component description underlying the enumeration in [4]; we recall its connector proof. Let X, Y be the left and right transversal endpoint sets of e , and let $p : X \rightarrow Y$ be the bijection specified by its transversals. At the middle interface of e^2 , retain the right caps of the first copy and the left caps of the second. Contracting the edges that identify equal labels gives the cap graph just defined. A missing blue edge is a right-transversal endpoint in Y , and a missing red edge is a left-transversal endpoint in X .

Because $\text{rank}(e^2) = \text{rank}(e)$, every endpoint of the first kind must connect to one of the second kind. A path with both endpoints of the same kind would join two transversals coming from the same outer side and reduce the rank. Thus each nontrivial path has an even number of alternating edges and an odd number of vertices. Its endpoints define a bijection $\gamma : Y \rightarrow X$; an isolated vertex in $X \cap Y$ is included as a connector of length zero. The transversal bijection of e^2 is $p\gamma p$. Since it equals p , one has $\gamma = p^{-1}$. Hence the transversal belonging to a connector path joins that path's endpoints, and a zero-length connector forces a fixed transversal. All remaining components are alternating cycles. The stated component contributions now follow by counting caps.

Conversely, for a matching assembled in this way, each interface connector is the inverse of its transversal. Gluing two copies therefore reproduces every transversal, while the outer caps already agree. Closed interface components do not affect the product. Thus the square equals the original diagram. $\square$

2.2 A budget for arbitrarily long words

Theorem 2.2 (Corank budget). *Let $e_1, \dots, e_v \in \mathcal{B}_m$ be idempotents. Every word w in these generators, allowing repetitions and the empty word, satisfies*

$$\text{corank}(w) \leq \sum_{i=1}^v \text{corank}(e_i). \quad (4)$$

Thus each chosen generator is charged once, independently of how often it occurs in the word.

Proof. Superimpose the cap graphs of the generators on $[m]$. Retain each edge occurrence, including occurrences with the same endpoints from different generators. Let C be a connected component of this union multigraph, and let E_C be its number of edges, counted with these multiplicities.

Each cap component of each individual e_i is entirely contained in a union component. By Lemma 2.1, it has an even number of edges. Consequently E_C is even. Connectedness gives $E_C \geq |C| - 1$, and parity improves this to

$$E_C \geq 2 \left\lfloor \frac{|C|}{2} \right\rfloor. \quad (5)$$

This includes isolated vertices, for which both sides vanish.

Every cap and every transversal of every e_i stays inside a union component, again by Lemma 2.1. The same is therefore true in any glued word and in the matching obtained after contraction. The restriction of w to the two copies of C is a Brauer diagram of degree $|C|$. Its rank has the parity of $|C|$, so its corank is at most $2\lfloor |C|/2 \rfloor$, and hence at most E_C . Summing over C proves the result, because the total number of cap edge occurrences is exactly $\sum_i \text{corank}(e_i)$. $\square$

Corollary 2.3 (Budget relative to an idempotent). *Let $e, b_1, \dots, b_v \in \mathcal{B}_m$ be idempotents. If w is any word in them and $z = ewe$, then*

$$\text{rank}(e) - \text{rank}(z) \leq \sum_{i=1}^v \text{corank}(b_i). \quad (6)$$

The conclusion applies to every positive power of every such sandwich, with the same right-hand side.

Proof. Apply Theorem 2.2 to the generating family $e, b_1, \dots, b_v$, then subtract $\text{corank}(e)$ from the resulting ambient corank inequality. Repeated sandwiches and their powers are still words in this fixed family. $\square$

Remark 2.4. The bound does not require disjoint supports or commuting generators. The hypothesis of idempotence supplies both facts used in the proof: an even cap-edge count on each individual component, and preservation of that component by transversals. Merely controlling the size of the union of supports loses this information. The coefficient one is sharp, already for a single nonidentity idempotent and the word consisting of that generator. Equality with several nonidentity generators is obtained by putting identical left and right cap matchings on disjoint even-sized blocks and using fixed transversals elsewhere: their product spends exactly the sum of the block sizes.

2.3 Sharp support, equality, and stability

The same classical components give an exact refinement of the support estimate used in [9]. This refinement is not needed to prove Theorem 2.2, but it describes precisely where that earlier support estimate wastes rank information.

Theorem 2.5 (Sharp support and its deficit). *Let $e \in \mathcal{B}_m$ be idempotent, let $c = \text{corank}(e)$, and set $\delta(e) = 3c/2 - |\text{supp}(e)|$. Then*

$$|\text{supp}(e)| \leq \min\{m, 3c/2\}. \quad (7)$$

For every admissible even $c \in [0, m]$, this is the exact maximum support among idempotents of corank c .

More precisely, if the nontrivial odd path components have sizes $2a_j + 1$ with $a_j \geq 1$, and the cycle components have sizes $2b_k$ with $b_k \geq 1$, then

$$\delta(e) = \sum_j (a_j - 1) + \sum_k b_k. \quad (8)$$

Equality $|\text{supp}(e)| = 3c/2$ holds exactly when all nontrivial components are three-vertex paths. Removing at most $5\delta(e)$ indices, in whole components, leaves only such paths and fixed transversals. The removed components contribute at most $4\delta(e)$ to the original corank. The constants 5 and 4 are both best possible for these component deletion statements.

Proof. By Lemma 2.1, the support is exactly the union of nontrivial cap components. A path of size $2a + 1 \geq 3$ contributes $2a$ to corank and $2a + 1$ to support, giving deficit $a - 1$. A cycle of size $2b$ contributes $2b$ to both, giving deficit b . Isolated vertices contribute zero. This proves (8), the bound, and the equality classification.

For attainability, write $c = 2q$. If $m \geq 3q$, use q disjoint three-vertex path components and $m - 3q$ isolated vertices. If $2q \leq m < 3q$, put $r = m - 2q$. Use r three-vertex paths and $q - r$ two-vertex cycles. These components use all m vertices, have rank r , and attain support m .

For stability, remove every cycle and every odd path with at least five vertices. A removed path has $a \geq 2$ and $2a + 1 \leq 5(a - 1)$; a removed cycle satisfies $2b \leq 5b$. Equation (8) bounds the total number of removed vertices by $5\delta(e)$. A single five-vertex alternating path has deficit one and must be removed in its entirety under the stated component deletion rule. Hence the factor cannot be smaller than five. The removed paths satisfy $2a \leq 4(a - 1)$ for $a \geq 2$, while the removed cycles satisfy $2b \leq 4b$. Their total corank is at most $4\delta(e)$. The same five-vertex path has corank four and deficit one, proving sharpness of this second constant. $\square$

Corollary 2.6 (Number of support maximizers). *For $q \geq 0$ and $m \geq 3q$, the number of labeled idempotents of corank $2q$ and support $3q$ is*

$$\frac{m!}{q!(m - 3q)!}. \quad (9)$$

Proof. Choose q unordered disjoint triples and leave the other vertices isolated. Their number is

$$\frac{m!}{(3!)^q q! (m - 3q)!}.$$

On each triple there are six idempotent hooks: choose the two distinct transversal endpoints in order; the remaining vertex determines both caps. Multiplying by 6^q gives the formula. $\square$

2.4 Corner reductions and unit lifts

We next record the finite-semigroup tools needed to apply the budget inside successive corners. A *unital* homomorphism preserves the designated identity, including when the identity of a corner differs from that of its ambient monoid.

Lemma 2.7 (Faithful reduction of a matching corner). *Let $e \in \mathcal{B}_m$ be idempotent and put $r = \text{rank}(e)$. There is an injective unital homomorphism*

$$R_e : e\mathcal{B}_m e \longrightarrow \mathcal{B}_r$$

such that $\text{rank}(R_e(z)) = \text{rank}(z)$ for every z in the corner.

Proof. Use the endpoint sets X, Y and transversal bijection $p : X \rightarrow Y$ from Lemma 2.1. Every $z = eae$ contains all the left caps of e on its outer left side and all the right caps of e on its outer right side. Delete those prescribed caps, retaining the left ports in X , the right ports in Y , and every matching edge among these retained ports. In particular, additional caps among retained ports are not deleted.

Number X by $[r]$ and give the right port $p(x)$ the same number as the left port x . The retained diagram defines $R_e(z)$. Restoring the prescribed caps reconstructs z , proving injectivity; no transversal was deleted, proving rank preservation.

At the interface of zw with $z, w \in e\mathcal{B}_m e$, the prescribed caps form precisely the connector of e^2 . The connector sends $p(x)$ to x , as established in Lemma 2.1. Contracting it therefore identifies equal numbers in the two reduced diagrams. Any closed connector components are irrelevant. This gives $R_e(zw) = R_e(z)R_e(w)$, and the numbering gives $R_e(e) = I_r$. If $r = 0$, all outer ports are prescribed caps, so the same argument identifies the corner with the one-element monoid $\mathcal{B}_0$. $\square$

This explicit construction is a version of the local-monoid methods used for diagram semigroups; compare [2, 12, 9].

Lemma 2.8 (Minimum-rank corners lift units). *Let $S \subseteq \mathcal{B}_m$ be nonempty and closed under multiplication, and let $\phi : S \rightarrow M$ be multiplicative, where M is a monoid. For each idempotent $q \in M$ there is an idempotent $e \in S$ above q . Every such e gives a unital surjection*

$$R_e(eSe) \twoheadrightarrow qMq, \quad R_e(z) \mapsto \phi(z). \quad (10)$$

If e has minimum rank among idempotents above q , every element of eSe whose image is a unit of qMq reduces to a permutation diagram. In particular all units of qMq have unit lifts in the reduced corner.

Proof. In any finite semigroup an element has a positive idempotent power: its powers are eventually periodic, and an exponent sufficiently large and divisible by the eventual period has this property. An idempotent power of a preimage of q is still above q . Furthermore, $\phi(eSe) = qMq$ by surjectivity, and the identity e of the corner maps to q . Lemma 2.7 transfers this map faithfully to the reduced corner, proving (10).

Now let $y \in eSe$ map to a unit. Some positive power of its image is q , and a further positive power of y is an idempotent z above q . Minimality and (3) give

$$\text{rank}(e) \leq \text{rank}(z) \leq \text{rank}(y) \leq \text{rank}(e).$$

Thus $R_e(y)$ has full rank and is a permutation. Its inverse is a positive power of itself and therefore belongs to $R_e(eSe)$. Surjectivity supplies such a preimage for every target unit. $\square$

For a finite set H , let $\mathcal{R}_H$ be the monoid of all binary relations on H , in path order: $(x, z) \in AB$ when $(x, y) \in A$ and $(y, z) \in B$ for some y . Its identity is I_H and its zero is the empty relation. Its units are exactly the permutation relations. A relation on $F \subseteq H$ is identified with the same subset of $H \times H$; equivalently, it is a relation A satisfying $A = I_F A I_F$.

Lemma 2.9 (A full relation corner). *Suppose $P \in \mathcal{R}_H$ is idempotent and $F \subseteq H$ satisfies $I_F P I_F = I_F$. Put $E = I_F$ and $Q = PEP$. Then Q is idempotent, $PQ = QP = Q$, and*

$$Q\mathcal{R}_H Q \longrightarrow \mathcal{R}_F, \quad Z \longmapsto EZE \quad (11)$$

is a unital isomorphism, with inverse $A \mapsto PAP$.

Proof. From $P^2 = P$ and $EPE = E$ one obtains $Q^2 = Q$, $PQ = QP = Q$, $EQ = EP$, $QE = PE$, and $EQE = E$. If $Z = QZQ$, then $PZP = Z$ and

$$Z = PEPZPEP = PEZEP.$$

Thus EZE determines Z . Conversely, for $A = EAE$,

$$QAZ = PAP, \quad E(PAP)E = A.$$

These formulas prove the inverse assertion. For $A, B \in \mathcal{R}_F$,

$$(PAP)(PBP) = PAPBP = PA(EPE)BP = PABP,$$

so the bijection respects products. It sends the corner identity Q to E , proving unitality. $\square$

2.5 Amplifying the rank cost of one relation addition

The next construction follows the two-stage relation-corner strategy of [9]. Its rank accounting is different: Corollary 2.3 controls both the complete chain and each local sandwich, without passing through support sizes.

Definition 2.10. A number $L \geq 0$ is an *addition bound at size $h \geq 2$* if the following holds. Whenever an identity-containing submonoid $S \subseteq \mathcal{B}_m$ admits a unital surjection $\phi : S \twoheadrightarrow \mathcal{R}_H$, with $|H| = h$ and with permutation diagram lifts for all permutations of H , every idempotent $a \in S$ with

$$\phi(a) = I_H \cup \{(x, y)\}, \quad x \neq y,$$

has $\text{corank}(a) \geq L$.

Proposition 2.11 (General amplification). *Let $p, q \geq 1$, $h \geq p + q + 1$, and $k = h - p - q + 1$. If L is an addition bound at size k , then*

$$\frac{pq}{p+q} L \quad (12)$$

is an addition bound at size h .

Proof. Fix data from Definition 2.10 at size h and write $c = \text{corank}(a)$. Choose disjoint sets $U = \{u_1, \dots, u_p\}$, $V = \{v_1, \dots, v_q\}$, and $\{z\}$ inside H . Conjugating a by permutation lifts supplies idempotents $s_i, t_j \in S$ such that

$$\phi(s_i) = I_H \cup \{(u_i, z)\}, \quad \phi(t_j) = I_H \cup \{(z, v_j)\}. \quad (13)$$

All these conjugates have corank c . The inverses of the chosen permutation lifts belong to S , since each is a power of a finite-order permutation.

Put $P_0 = I_{H \setminus \{z\}}$ and choose an idempotent lift $b_0 \in S$ of P_0 by taking a positive idempotent power. For each pair (i, j) let $g_{ij} = b_0 s_i t_j b_0$. The path through z in (13), followed by deletion of its endpoints from the outer relation, gives

$$\phi(g_{ij}) = P_0 \cup \{(u_i, v_j)\}.$$

List the pq pairs in any order. If $E_\ell \subseteq U \times V$ contains the first ℓ listed pairs, define $P_\ell = P_0 \cup E_\ell$. Because $U \cap V = \emptyset$,

$$(P_0 \cup A)(P_0 \cup B) = P_0 \cup (A \cup B) \quad (A, B \subseteq U \times V). \quad (14)$$

In particular all P_ℓ are idempotent.

Inductively choose b_ℓ to be a positive idempotent power of $b_{\ell-1} g_{ij} b_{\ell-1}$, where (i, j) is the ℓ th pair. Equation (14) gives

$$b_\ell^2 = b_\ell, \quad b_\ell \in b_{\ell-1} S b_{\ell-1}, \quad \phi(b_\ell) = P_\ell. \quad (15)$$

Write $r_\ell = \text{rank}(b_\ell)$ and $\Delta_\ell = r_{\ell-1} - r_\ell \geq 0$. Every b_ℓ is a word in the fixed idempotent family $b_0, s_1, \dots, s_p, t_1, \dots, t_q$ and lies in the corner of b_0 . Corollary 2.3 therefore gives the single global budget

$$\sum_{\ell=1}^{pq} \Delta_\ell = r_0 - r_{pq} \leq (p+q)c. \quad (16)$$

We prove that each $\Delta_\ell \geq L$. Fix the pair (i, j) at step ℓ , put $b = b_{\ell-1}$ and $P = P_{\ell-1}$, and pass to the reduced corner

$$T = R_b(b S b) \subseteq \mathcal{B}_{r_{\ell-1}}, \quad \beta = R_b(b_\ell).$$

The induced map $\theta : T \rightarrow P \mathcal{R}_H P$ is unital by Lemma 2.8, and β is idempotent with

$$\theta(\beta) = P_\ell, \quad \text{corank}(\beta) = \Delta_\ell.$$

Here and below corank is computed in the currently displayed ambient Brauer monoid; all ranks are preserved by the reductions.

Retain precisely

$$F = \{u_i, v_j\} \cup (H \setminus (U \cup V \cup \{z\})), \quad E = I_F.$$

Then $|F| = k$. The only potential earlier added pair with both endpoints in F is (u_i, v_j) , and this pair has not yet been added. Thus $E P E = E$. Put $Q = P E P$. By Lemma 2.9, restriction identifies $Q \mathcal{R}_H Q$ with $\mathcal{R}_F$, and

$$Q(P \mathcal{R}_H P)Q = Q \mathcal{R}_H Q.$$

Moreover, $P P_\ell = P_\ell P = P_\ell$, so

$$E(Q P_\ell Q)E = E P_\ell E = I_F \cup \{(u_i, v_j)\}. \quad (17)$$

Choose an idempotent $d \in T$ of minimum rank among those with $\theta(d) = Q$, and set $r' = \text{rank}(d)$. Lemma 2.8 and the restriction isomorphism give a unital surjection

$$\psi : T' = R_d(d T d) \rightarrow \mathcal{R}_F$$

with permutation diagram lifts for every permutation of F . Take a positive idempotent power v of $R_d(d\beta d)$. Equation (17) says that its image before taking the power is already the idempotent relation $I_F \cup \{(u_i, v_j)\}$; the power has the same image. The size- k addition bound consequently gives $\text{corank}(v) \geq L$.

On the other hand, apply Corollary 2.3 to the two idempotents d, β in $\mathcal{B}_{r_{\ell-1}}$. The corresponding power of $d\beta d$ loses at most $\text{corank}(\beta)$ relative to $\text{rank}(d)$. Faithfulness and rank preservation of R_d give

$$L \leq \text{corank}(v) \leq \text{corank}(\beta) = \Delta_\ell.$$

This proves the local claim. Summing it and using (16) yields $pqL \leq (p+q)c$, as required. $\square$

Theorem 2.12 (Exponential cost of an addition). *For every $h \geq 2$, the number*

$$F(h) = 2 \left(\frac{5}{2} \right)^{\lfloor (h-2)/9 \rfloor} \quad (18)$$

is an addition bound at size h .

Proof. Any idempotent above a nonidentity relation has positive corank: otherwise it is the ambient identity, contradicting unitality. Corank is even, so it is at least two. This proves the assertion for $2 \leq h \leq 10$. For $h \geq 11$, use strong induction and Proposition 2.11 with $p = q = 5$. It gives an addition bound $(5/2)F(h-9) = F(h)$. All smaller sets used in this step have size $h-9 \geq 2$. $\square$

Theorem 2.13 (Degree of a full relation-monoid divisor). *Let H have $h \geq 2$ elements. If an identity-containing submonoid $S \subseteq \mathcal{B}_k$ has a unital surjection onto $\mathcal{R}_H$, then*

$$k \geq F(h) = 2 \left(\frac{5}{2} \right)^{\lfloor (h-2)/9 \rfloor}. \quad (19)$$

No assumption about permutation lifts is needed.

Proof. Choose an idempotent $e \in S$ of minimum rank above I_H ; the identity of S is one possible choice. Its reduced corner $R_e(eSe) \subseteq \mathcal{B}_r$, where $r = \text{rank}(e)$, maps unittally onto $\mathcal{R}_H$ and has permutation lifts by Lemma 2.8. Lift any relation $I_H \cup \{(x, y)\}$ with $x \neq y$ to that corner and take an idempotent power. Theorem 2.12 bounds its corank below by $F(h)$, whereas its corank is at most $r \leq k$. $\square$

2.6 Choice of parameters and finite integer bounds

Proposition 2.14 (Optimal fixed rectangular amplification). *Among fixed positive integer choices of p, q , the ratio*

$$\frac{\log(pq/(p+q))}{p+q-1} \quad (20)$$

is maximized at $p = q = 5$. Thus the fixed choice in Theorem 2.12 has the best asymptotic exponential rate within Proposition 2.11.

Proof. At fixed $p+q$, balancing p, q maximizes their product. For an even sum write $p = q = t$; the rate is $f(t) = \log(t/2)/(2t-1)$. Its derivative has the sign of

$$N(t) = 2 - 1/t - 2\log(t/2), \quad N'(t) = (1-2t)/t^2 < 0 \quad (t > 1/2).$$

One has $N(4) > 0 > N(5)$: for example, $\log 2 < 3/4$ and $\log(5/2) > 312/343 > 9/10$, using the first two positive terms of $\log((1+x)/(1-x))$ at $x = 3/7$ for the latter bound. Thus only $t = 4, 5$

can maximize the rate over integers. The inequality $f(5) > f(4)$ is equivalent to $5^7 > 2^{16}$, namely $78125 > 65536$.

For an odd sum $2t + 1$, the best factor is $t(t + 1)/(2t + 1)$, and

$$\frac{t(t + 1)}{2t + 1} \leq \sqrt{\frac{t}{2} \frac{t + 1}{2}}.$$

Its deletion cost $2t$ is the arithmetic mean of $2t - 1$ and $2t + 1$. Taking logarithms shows that its rate is at most the larger of the rates for the two neighboring even sums. This completes the optimization. $\square$

The resulting base-two rate is

$$\frac{\log_2(5/2)}{9} = 0.1468808994 \dots$$

The optimization concerns this particular rectangular construction; it is not an optimality assertion about the true minimum degree of a Brauer monoid admitting the required divisor.

Proposition 2.15 (Parity-aware finite recurrence). *Define positive integers $D(h)$ recursively by $D(2) = 1$ and, for $h \geq 3$,*

$$D(h) = \max \left\{ 1, \max_{2 \leq s \leq h-1} \left\lceil \frac{\lfloor s^2/4 \rfloor}{s} D(h - s + 1) \right\rceil \right\}. \quad (21)$$

Then $2D(h)$ is an addition bound at size h , and the conclusion of Theorem 2.13 can be strengthened to $k \geq 2D(h)$.

Proof. Use strong induction. For each allowed s , choose balanced positive p, q with $p + q = s$ and $pq = \lfloor s^2/4 \rfloor$. Proposition 2.11 and the induction hypothesis give corank at least $2pqD(h - s + 1)/s$. Corank is an even integer, so it is at least twice the corresponding ceiling in (21). The baseline bound is two, and taking the maximum proves the addition claim. The minimum-rank corner proof of Theorem 2.13 then applies unchanged. Every recursive argument $h - s + 1$ lies between 2 and $h - 1$; no monotonicity of D is assumed or needed. $\square$

[write] **Checked, 7 October 2026.** Read line by line, with no error found. Rechecked by hand: the connector argument of Lemma 2.1 ($\gamma = p^{-1}$ from $p\gamma p = p$); the parity step $E_C \geq 2\lfloor |C|/2 \rfloor$ and the restriction argument of Theorem 2.2, and the subtraction giving Corollary 2.3; the deficits $a - 1$ and b , the attaining configurations ($r = m - 2q$ three-vertex paths and $q - r$ two-cycles when $2q \leq m < 3q$) and the inequalities $2a + 1 \leq 5(a - 1)$, $2a \leq 4(a - 1)$ for $a \geq 2$ of Theorem 2.5; the six hooks per triple of Corollary 2.6; the identities $Q^2 = Q$, $PQ = QP = Q$, $EQE = E$, $QEQ = PAP$ and $(PAP)(PBP) = PABP$ of Lemma 2.9; in Proposition 2.11 the image $P_0 \cup \{(u_i, v_j)\}$ of g_{ij} , the global budget (16) from Corollary 2.3 with $e = b_0$ and the $p + q$ conjugates, $|F| = h - p - q + 1$, and $EPE = E$; the base range $2 \leq h \leq 10$ and the step $(5/2)F(h - 9) = F(h)$ of Theorem 2.12; in Proposition 2.14, $2(x + x^3/3) = 312/343$ at $x = 3/7$, $N(4) > 0 > N(5)$, $5^7 = 78125 > 65536 = 2^{16}$ and the mediant step for odd sums; and the evenness step of Proposition 2.15. The rate $\log_2(5/2)/9 = 0.1468808994 \dots$ is a correct truncation (next digits 3192). In Proposition 2.11 the letter k is a set size, and in Proposition 2.15 the letter s is an arm sum, not a number of states (reading conventions, Section 1.5).

3 A linear-degree local representation of deterministic machines

The connection with automata requires more than a matching representation of a completed configuration graph. Each input symbol must determine its own diagram, independently of its neighbors and its position. We adapt the candidate-slot construction of [9], retaining only crossing transitions that occur somewhere in the finite transition table. This changes its quadratic degree bound to a linear bound for every fixed alphabet. The underlying traversal of a configuration tree is related to the halting and reversible-simulation methods of [17, 7]; all traversal and locality claims needed here are proved below.

3.1 Machine convention and statement

Let Σ be an alphabet of size a . The scanned input is $\vdash w \dashv$, where the two endmarkers are distinct and are not in Σ . An s -state deterministic two-way finite automaton has state set Q , initial state q_0 , accepting set $A \subseteq Q$, and a partial function

$$\delta : Q \times (\Sigma \cup \{\vdash, \dashv\}) \rightarrow Q \times \{-1, 0, 1\}.$$

Moves at endmarkers never leave the marked tape. The initial head position is the left marker. Acceptance means that an accepting state occurs at some finite time, including time zero. An undefined transition halts; an infinite run without an accepting state rejects. Initial and accepting states are included in s . Stay moves, partial transition tables, and rejecting cycles are allowed throughout.

Theorem 3.1 (Sparse-slot local representation). *For every machine in this convention, there are an even integer k and a map*

$$\eta : \Sigma \cup \{\vdash, \dashv\} \rightarrow \mathcal{B}_k$$

such that

$$2 \leq k \leq 2(a+2)s+2. \quad (22)$$

Designate ports 1, 2 on each side as test ports. For every $w = a_1 \cdots a_\ell$, the machine accepts w exactly when

$$\eta(\vdash)\eta(a_1) \cdots \eta(a_\ell)\eta(\dashv) \quad (23)$$

pairs a left test port with a right test port. In that case both left test ports are paired with right test ports. Each diagram depends only on its displayed symbol and the machine. In particular, for $\Sigma = \{0, 1\}$ one has $k \leq 8s+2$.

3.2 Acceptance becomes reachability of one sink

Add a new state f and define an auxiliary partial successor rule by

$$\widehat{\delta}(q, b) = \begin{cases} (f, 0), & q \in A, \\ \delta(q, b), & q \in Q \setminus A, \\ (f, 1), & q = f, b \neq \dashv, \\ \text{undefined}, & q = f, b = \dashv. \end{cases} \quad (24)$$

The accepting-state rule takes precedence even when the original entry was undefined. For a fixed word of length ℓ , let $c_0 = (0, q_0)$ and $t = (\ell+1, f)$. The original machine accepts exactly when this new successor rule reaches t from c_0 . Until an accepting state first occurs, the runs agree. Such

a state enters f , which sweeps to the right marker and stops. Conversely, the first entry into f must come from an old accepting state. This includes acceptance initially and acceptance on either marker. There is no additional copy of the initial state.

Lemma 3.2 (The component of a functional sink). *Let $\sigma : V \rightarrow V$ be a partial function on a finite set, and suppose $\sigma(t)$ is undefined. Make an undirected multigraph with one edge for each defined successor, retaining loops and parallel edges. The component of t consists precisely of the vertices whose forward successor paths reach t , and this component is a tree.*

Proof. Write B for the set of vertices reaching t , allowing zero steps. For every successor $u \mapsto v$, one has $u \in B$ if and only if $v \in B$: prepend the edge in one direction, and remove the first step of the unique successor path in the other. Here $u \neq t$, because t has no successor. Membership in B is therefore constant along undirected edges. A forward path is also an undirected path, so B is exactly the component of t .

Every vertex of that component other than t has exactly one outgoing edge, and all these edges stay in the component. If the component has v vertices, it consequently has $v - 1$ edges, counted with multiplicity. It is connected and hence is a tree, with neither loops nor parallel edges. $\square$

Apply the lemma to all configurations of (24). Call the resulting undirected successor multigraph G . Its component containing t is a tree, and contains c_0 precisely on accepted inputs. Other components may contain nonaccepting cycles. They need no modification.

3.3 A sparse set of boundary slots

Put $Q' = Q \cup \{f\}$. The fixed set of crossing candidates is

$$C = \{(d, p, q) : d \in \{-1, 1\}, p, q \in Q', \hat{\delta}(p, b) = (q, d) \text{ for some } b\}, \quad J = |C|. \quad (25)$$

Each old state supplies at most $a + 2$ candidates, one per scanned symbol. The new state supplies just the single candidate $(1, f, f)$, since all its defined crossing rules are identical. Therefore

$$1 \leq J \leq (a + 2)s + 1, \quad k = 2J \leq 2(a + 2)s + 2. \quad (26)$$

In fact the first upper bound can be improved to $(a + 2)(s - |A|) + 1$, because old accepting states now have only stay rules. The important point is that the candidate set is a union over local transition-table entries, not over all pairs of states.

Fix an enumeration of C . At every boundary between neighboring cells, reserve one edge slot for each $(d, p, q) \in C$. The direction d determines which cell is the source: the left cell for $d = 1$, the right cell for $d = -1$. At its target end the candidate edge is always attached to state vertex q . At its source end attach it to state vertex p if that cell's scanned symbol makes $\hat{\delta}(p, b) = (q, d)$. Otherwise attach it to a fresh degree-one vertex. Different unused slots have different fresh vertices.

Every actual crossing successor edge occurs in its own slot. Every other slot is a leaf edge attached to a state vertex at its target end. Keep every actual stay edge inside its cell, with two distinct incidences when the edge is a loop. The resulting graph is exactly G with additional leaves and leaf edges. Thus it preserves connectivity between configurations and preserves the tree property of the component containing t .

This asymmetric attachment rule is what ensures locality. A target cell never tests its neighbor's symbol: its target attachment is unconditional. Only the source cell chooses between a state vertex and a fresh leaf, using its own transition row. Omitting candidates that never occur removes possible dummy leaves, without removing any actual successor.

3.4 Contour wires and the two marker tests

An incidence records one endpoint of an edge at its vertex. A loop has two distinct incidences at the same vertex. At every vertex of positive degree choose a cyclic order of its incidences. An arrival along one incidence is followed by departure along the next incidence in this order. A degree-one vertex consequently reflects the traversal back along its edge.

Lemma 3.3 (Contour of a tree). *On a finite tree with at least one edge, these successor rules form one cycle on the directed edge traversals, for every choice of cyclic orders. Opening the reflection joins at two distinct leaves produces two paths, each joining a tip at one leaf to a tip at the other.*

Proof. The rule permutes the directed traversals: every departure has one preceding arrival. A one-edge tree has a single cycle of length two. For induction, remove a leaf z from a larger tree and call its neighbor v . Restrict the cyclic order at v to the remaining incidences. Reinserting vz replaces one step of the smaller contour by a detour from v to z , back to v , and then onward. It inserts two directed traversals into the existing cycle and creates no new cycle. The contour is therefore a circular wire. Cutting it at two distinct reflection joins produces two intervals, each with one endpoint at each cut. $\square$

Only inward boundaries of marker cells receive candidate slots. On the outward side of the left-marker cell, add a test edge from c_0 to a new outer leaf z_L . On the outward side of the right-marker cell, add a test edge from t to another new outer leaf z_R . These two test edges preserve the tree property of the t -component. The test leaves are connected if and only if the original machine accepts.

Each graph edge now receives two lanes, one in each travel direction. An incidence has an incoming tip and an outgoing tip. For an edge with incidences i, j , one lane joins the outgoing tip at i to the incoming tip at j , and the other joins the outgoing tip at j to the incoming tip at i . This also applies to loops, whose incidences remain distinct. At every vertex except z_L, z_R , join the incoming tip of each incidence to the outgoing tip of the next incidence in the chosen cyclic order. The two outer test-leaf joins are left open.

For each candidate slot, its two ports are labeled by absolute travel directions $\rightarrow, \leftarrow$. The same enumeration is used on both sides of every cell. The correspondence of ports to local tips is

cell side	$\rightarrow$ lane	$\leftarrow$ lane
left	incoming	outgoing
right	outgoing	incoming

Consequently equal-index gluing joins an outgoing half-lane in one cell to the appropriate incoming half-lane in the next.

At each outward marker side, use ports 1, 2 for the two open tips of its test edge. The same indices remain ordinary candidate-lane indices on inward and ordinary boundaries. Pair the remaining outward ports as $(3, 4), (5, 6), \dots, (k-1, k)$, disconnected from all graph edges. This uses every outward port because k is even; for $k = 2$ there are no remaining ports.

Choose incidence orders separately for each symbol type, then reuse those orders at every occurrence of that symbol. For example, order incidences by boundary side and candidate index, followed by state-indexed stay-edge roles and the marker test role. A loop's two roles are distinct. Every incidence and its attachment are determined by the local symbol, so this choice requires no information about an adjacent cell.

3.5 Local contraction and correctness

Inside one cell, every lane tip belongs to one lane and one join. The only exposed endpoints are its boundary ports, each used exactly once. The separately paired unused outward ports have the same degree-one property. The finite local wiring therefore has degree two at internal points and degree one at its ports. Each component containing a port is a path to exactly one other port. Discard closed components and pair the two endpoints of every remaining path. This defines a perfect matching on the cell's $2k$ ports, which is the required $\eta(b)$.

All operations used to define this matching depend only on b and the machine. In particular, an ordinary symbol has the same diagram at its first, last, and intermediate occurrences. Gluing equal-index ports of neighboring cell diagrams reconstructs the full two-lane wiring. Contracting paths before gluing or after gluing yields the same matching of outer ports, by the path definition of Brauer multiplication. Thus (23) represents the complete wiring with the two test-leaf joins open.

If a wire joins a left test port to a right test port, it stays inside one component of the augmented graph. The separately paired outward ports cannot participate. Hence z_L and z_R are connected, and the machine accepts. Conversely, on an accepted input both test leaves lie in the tree component of t . Temporarily close their reflection joins. Lemma 3.3 gives one circular contour through both joins; opening them again gives two paths between the two test sides. Both left test ports therefore meet right test ports. This proves the acceptance criterion and completes Theorem 3.1.

For w empty, there are still two distinct adjacent marker cells. Their inward boundaries glue by precisely the same slot scheme, and both test edges are present. The accepted component has at least those two edges, so the tree-contour argument applies without an exceptional case. Rejecting cycles elsewhere never enter the proof of the converse and cannot create a wire between disconnected test leaves.

Extend η to body words by multiplication, with $\eta(\varepsilon) = I$. Equality $\eta(u) = \eta(v)$ then implies that xuy and xvy have the same acceptance status for every pair of body words x, y . This follows by adjoining the same marker diagrams and context diagrams to the two equal products. This context property is the precise form needed for the syntactic-monoid arguments below.

3.6 Why the linear order cannot be improved universally

The following observation concerns arbitrary recognition by Brauer monoids, so it also applies to the particular test-port representation. Fixed marker diagrams and a fixed port predicate can always be absorbed into an accepting subset of the monoid of body-word diagrams.

Theorem 3.4 (A linear degree is sometimes necessary). *For every $s \geq 2$ there is a binary language recognized by an s -state one-way DFA under the usual end-of-input acceptance convention such that every monoid recognizer*

$$\rho : \{0, 1\}^* \longrightarrow \mathcal{B}_m, \quad L = \rho^{-1}(P), \quad P \subseteq \mathcal{B}_m,$$

has $m \geq s$. The same language has an $(s+1)$ -state machine in the accepting-at-any-time convention of this section. Consequently a universal degree bound of order $o(s)$ is impossible even over a binary alphabet.

Proof. Use state set $[s]$, initial state 1, and final set $\{1\}$. Let 0 act as the cycle $(1\ 2 \cdots s)$ and 1 as the transposition $(1\ 2)$. These permutations generate the full symmetric group $\mathfrak{S}_s$: conjugates of the transposition give the adjacent transpositions. Finite orders ensure that inverses are represented by positive words as well.

The syntactic monoid of the accepted language is precisely $\mathfrak{S}_s$. Equal word permutations give equal acceptance in every context. Conversely, suppose two permutations disagree at state i . A prefix can carry state 1 to i , and a suffix can carry one of the two distinct resulting states to 1 and the other away from 1. The corresponding context distinguishes the two words. This proves the claimed syntactic monoid directly.

Let $S = \rho(\{0, 1\}^*)$. If two words have equal ρ -images, every context also has equal images and hence equal acceptance. Therefore the assignment from a word image to its syntactic class is a well-defined unital surjection $S \twoheadrightarrow \mathfrak{S}_s$. Choose an idempotent e of minimum rank above the group identity and put $r = \text{rank}(e)$. Lemmas 2.7 and 2.8 give a reduced corner

$$R_e(eSe) \subseteq \mathcal{B}_r \quad \text{surjecting onto } \mathfrak{S}_s.$$

Every target element is a unit, so every element of this reduced corner is a permutation. Thus it is a subgroup of $\mathfrak{S}_r$ and

$$s! \leq |R_e(eSe)| \leq r! \leq m!.$$

For $s \geq 2$ this forces $m \geq s$.

To realize the witness in the convention used above, keep all s permutation states nonaccepting, move right on body symbols, and add one accepting state entered at the right marker exactly from state 1. The left-marker rule enters the body without changing the permutation state. Other right-marker rules may be undefined. This uses $s + 1$ states, including the new accepting state, and also handles the empty word. It proves the stated asymptotic necessity. The argument does not claim that the constant 8 in the binary upper bound is optimal. $\square$

[write] **Checked, 7 October 2026.** Read line by line, with no error found. Rederived by hand: the normalization (24) and its acceptance equivalence; Lemma 3.2 (a component with v vertices and $v - 1$ successor edges); the slot count $J \leq (a + 2)s + 1$, at most one crossing candidate per state and scanned symbol plus the single candidate $(1, f, f)$; the induction of Lemma 3.3; the acceptance criterion; and Theorem 3.4 ($s! \leq |R_e(eSe)| \leq r! \leq m!$). The lane wiring and its locality were read, not rebuilt independently; the shipped `code/sparse_slots_audit.py` builds the local diagrams for 800 machines and passed again at the write (Section 6).

4 A two-register compiler and the binary lower bound

The relation alphabet has 2^{h^2} symbols. We now replace it by a single fixed binary alphabet while keeping the nondeterministic source linear in h . A direct monoid argument transfers the relation information; it avoids charging a deterministic target for remembering a position inside a long encoded block.

4.1 Four letters on two cyclic registers

For this section index the h relation points by $\mathbb{Z}/h\mathbb{Z}$, using representatives $0, \dots, h - 1$. Relabeling has no effect on the results of Section 2. Take disjoint state sets

$$U = \{U_0, \dots, U_{h-1}\}, \quad V = \{V_0, \dots, V_{h-1}\}.$$

Define four relations on $U \cup V$:

$$a = \{(U_i, U_{i+1}) : i \in \mathbb{Z}/h\mathbb{Z}\} \cup I_V, \tag{27}$$

$$b = I_U \cup \{(V_j, V_{j+1}) : j \in \mathbb{Z}/h\mathbb{Z}\}, \tag{28}$$

$$t = I_{U \cup V} \cup \{(U_0, V_0)\}, \tag{29}$$

$$r = \{(V_j, U_j) : j \in \mathbb{Z}/h\mathbb{Z}\}. \tag{30}$$

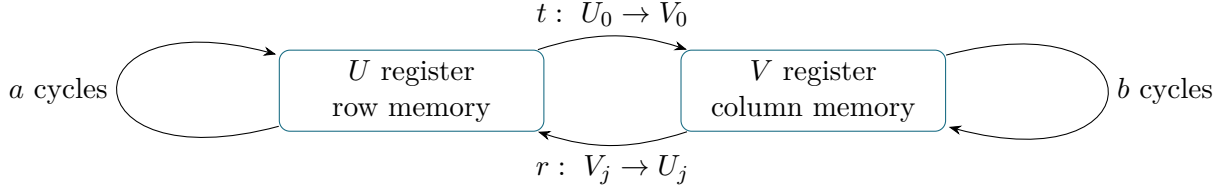


Figure 1: The four fixed relations. Identity edges of a, b, t are specified in (27)–(29). The final r has no edge out of U , forcing exactly one successful crossing per block.

The letter r is undefined on every state of U . Each of a, b, r is partial deterministic; the only extra branch is the edge $U_0 \rightarrow V_0$ of t .

For an h -point relation R , let $R(x, y) \in \{0, 1\}$ be its matrix entry. Products below are read from left to right in the displayed increasing order. Define

$$w(R) = \left[\prod_{i=0}^{h-1} \left(\left[\prod_{j=0}^{h-1} t^{R(-i, -j)b} \right] a \right) \right] r. \quad (31)$$

Here t^0 means the empty word and t^1 means the one-letter word. The negative indices are reduced modulo h .

Theorem 4.1 (Exact two-register compilation). *For every $x, y \in \mathbb{Z}/h\mathbb{Z}$, there is exactly one path from U_x to U_y labeled $w(R)$ if $(x, y) \in R$, and there is no such path otherwise. Every path starting in U and surviving the complete word ends in U . Furthermore,*

$$|w(R)| = h^2 + h + 1 + |R| \leq 2h^2 + h + 1. \quad (32)$$

Proof. Before crossing to V , a path starting at U_x is at U_{x+i} during row i : each completed row contributes one a , and all b letters fix U . It can use the additional branch of t only in the unique row $i = -x$. Within that row it can cross at column j exactly when the displayed exponent $R(-i, -j)$ is one.

After crossing it is at V_0 . The b immediately following that t , and the remaining b letters in the row, move it $h - j$ places, leaving it at V_{-j} . Each later complete row contains h copies of b , so those rows have no further net effect on its V index; their a letters fix V . All later t letters also fix that state. The final r sends it to U_{-j} . Thus a crossing in column $j = -y$ exists exactly when $R(x, y) = 1$.

The row and column of such a crossing are unique. At every other t letter the path must choose the identity edge. There is therefore exactly one successful path for each entry equal to one. A path which never crosses stays in U and is killed by r . No surviving path ends in V , since all edges of r end in U . Finally the word has h^2 copies of b , h copies of a , one r , and $|R|$ copies of t , proving (32). $\square$

Corollary 4.2 (Concatenations and multiplicity). *On the boundary states U , the relation of $w(R_1) \cdots w(R_\ell)$ is exactly the Boolean product $R_1 \cdots R_\ell$. More strongly, its matrix of numbers of paths between U states is the ordinary nonnegative-integer matrix product of the zero-one matrices of the R_i .*

Proof. Theorem 4.1 identifies each single-block path-count matrix on U with the corresponding zero-one matrix. Every surviving path is in U at each block boundary, so concatenation uses the

usual sum over intermediate states. The empty concatenation gives the identity matrix. Taking the support of the integer product gives the Boolean product. $\square$

The exact path-count statement is stronger than the language equivalence needed below. It provides a useful independent test: spurious duplicate branches cannot hide behind Boolean reachability. It also makes clear that the full NFA need not be unambiguous. Concatenations can have several intermediate-state paths between the same endpoints.

4.2 A complete binary language with $6h + 2$ states

Use the fixed code

$$a \mapsto 00, \quad b \mapsto 01, \quad t \mapsto 10, \quad r \mapsto 11, \quad (33)$$

and let $\text{enc}(R) = \text{code}(w(R))$. For each $q \in U \cup V$ take three decoder states

$$(q, \varepsilon), \quad (q, 0), \quad (q, 1).$$

From (q, ε) the first bit d goes to (q, d) ; from (q, d) the second bit e executes the four-letter relation corresponding to de and returns to a boundary state (q', ε) . All these transitions move right.

Add one initial state q_{in} and one accepting state q_{acc} . On $\vdash$, q_{in} guesses any (U_i, ε) and moves right. At $\dashv$, each (U_i, ε) has a stay move to q_{acc} . All other endmarker transitions are undefined, and q_{acc} has no successors. No other state is accepting.

This specifies a language B_h on *every* binary string. An odd-length string ends in a pending-bit state and rejects. An even-length string is decoded in consecutive two-bit groups; it accepts precisely when some resulting four-letter path starts and ends in U . There is no promise that the word consists of canonical relation blocks and no unstated treatment of malformed blocks. Some noncanonical even-length words may accept, according to this explicit transition table. The empty binary word accepts.

Proposition 4.3 (Binary state and length counts). *The specified automaton is a $6h + 2$ -state 1NFA and, for every finite sequence of h -point relations,*

$$\text{enc}(R_1) \cdots \text{enc}(R_\ell) \in B_h \iff R_1 \cdots R_\ell \neq 0. \quad (34)$$

The canonical block length is

$$|\text{enc}(R)| = 2(h^2 + h + 1 + |R|) \leq 4h^2 + 2h + 2.$$

Proof. There are $2h$ relation states with three decoder copies each, plus the two explicitly added states. The decoder executes exactly one four-letter transition per two input bits. On a canonical concatenation, Corollary 4.2 gives precisely the relation product on the U boundary states. The initial and final rules test whether this relation has any pair. This includes $\ell = 0$, since I_U is nonempty. The length formula follows from Theorem 4.1. $\square$

The order of the block length cannot be improved for an injective binary encoding of all relations. There are 2^{h^2} distinct relations, but only $2^{L+1} - 1$ binary strings of length at most L . Thus the largest block length of any such encoding is at least h^2 . In this sense the quadratic maximum length is optimal up to a constant factor. This counting observation does not imply optimality of the number of compiler states.

4.3 Singleton contexts force the full relation divisor

Let $\eta : \{0, 1\}^* \rightarrow M$ be a monoid homomorphism into any monoid which recognizes B_h : assume there is a set $A \subseteq M$ such that $w \in B_h$ exactly when $\eta(w) \in A$. This includes recognition by ordinary-symbol matching diagrams, since the two endmarker diagrams and the test ports define the set A .

Let $S \subseteq M$ be the submonoid generated by $\{\eta(\text{enc}(R)) : R \in \mathcal{R}_{[h]}\}$. It contains the empty product, with the same identity as M . We only require a relation quotient on S , not on all of M .

Lemma 4.4 (Encoded context separation). *There is a well-defined unital surjective homomorphism*

$$\phi : S \rightarrow \mathcal{R}_{[h]}, \quad \eta(\text{enc}(R_1) \cdots \text{enc}(R_\ell)) \mapsto R_1 \cdots R_\ell.$$

Proof. Suppose two canonical concatenations have the same element of M , but their relation products R and T differ. Choose (i, j) in their symmetric difference and put $E_i = \{(i, i)\}$ and $E_j = \{(j, j)\}$. Then

$$E_i R E_j \neq 0 \iff (i, j) \in R,$$

and the corresponding equivalence holds for T . Surround each canonical concatenation by $\text{enc}(E_i)$ on the left and $\text{enc}(E_j)$ on the right. By (34), precisely one resulting binary word belongs to B_h . Yet multiplicativity of η makes their images equal, contradicting recognition. This proves well-definedness, including comparisons with an empty concatenation.

Concatenating representatives proves multiplicativity. The empty word is sent to $I_{[h]}$, so the map is unital. Every relation is the image of its single canonical block, proving surjectivity. $\square$

This is the usual singleton-context syntactic argument, specialized to the encoded submonoid; compare [12, 9]. A decoder need not label every arbitrary binary word by an h -point relation in a multiplicative way. The narrower generated submonoid is both sufficient and necessary for the argument as stated.

4.4 Proof and quantitative comparison

Proof of Theorem 1.1. Take the binary language from Proposition 4.3 and an equivalent s -state 2DFA. By Theorem 3.1, its ordinary-symbol matching diagrams lie in $\mathcal{B}_k$ for some $k \leq 8s + 2$ and recognize B_h with fixed endmarker diagrams and fixed test ports. Lemma 4.4 supplies an identity-containing submonoid with a unital surjection onto $\mathcal{R}_{[h]}$. Theorem 2.13 therefore gives $8s + 2 \geq k \geq F(h)$, proving (1).

For $n \geq 14$, put $h = \lfloor (n - 2)/6 \rfloor \geq 2$. The source has $6h + 2 \leq n$ states and may be padded with unreachable, nonaccepting states to reach exactly n . Integer floor arithmetic gives

$$\left\lfloor \frac{\lfloor (n - 2)/6 \rfloor - 2}{9} \right\rfloor = \left\lfloor \frac{n - 14}{54} \right\rfloor.$$

Solving (1) for s proves (2). The exponential asymptotic follows by absorbing the floor and fixed multiplicative factors into $O(1)$. $\square$

The integer recurrence of Proposition 2.15 gives the useful finite refinement

$$s \geq \max \left\{ 1, \left\lceil \frac{D(h) - 1}{4} \right\rceil \right\}. \quad (35)$$

The additional 1 simply records that a recognizer has at least one state. The supplied recurrence script computes $D(h)$ using exact integers; no floating-point rounding enters (35).

Quantity	Inspected earlier construction	This article
Relation-divisor degree	$2^{\lfloor (h-2)/31 \rfloor}$	$2(5/2)^{\lfloor (h-2)/9 \rfloor}$
2DFA matching degree	$4(s+1)^2$ for arbitrary alphabets	$2(a+2)s+2$ for a letters
Binary matching degree	general bound $4(s+1)^2$	$8s+2$
Four-letter relation compiler	$3h$ states in R816's convention	$2h$ relation states
Binary source count	$9h$ states in R816's convention	$6h+2$ including our marker states
Binary conclusion	constant-factor transfer applies to an imported liveness bound	explicit bound (2)

Table 1: Comparison with the source matching argument [9] and the earlier encoding record [18]. Machine conventions explain the two added marker states in the present source count. The general quadratic port bound can be preferable when the alphabet is very large.

The leading exponential rate in the algebraic degree estimate increases from $1/31$ to $\log_2(5/2)/9$ when measured in h . The sparse matching representation then avoids taking a square root of that degree lower bound for a fixed alphabet. These are separate improvements; the binary encoding permits both to contribute to a bound measured in the number of source states. The lower bound concerns states, not running time, number of head reversals, or the length of a transition-table description.

[write] Checked, 7 October 2026. Read line by line, with no error found. Rederived by hand: the path analysis of Theorem 4.1 (a path from U_x sits at U_{x+i} in row i , can cross only in row $i = -x$, reaches $V_{h-j} = V_{-j}$ by the end of that row and U_{-j} after r) and the length $h^2 + h + 1 + |R|$; the state count $3 \cdot 2h + 2 = 6h + 2$; the counting bound on block length ($2^{L+1} - 1 \geq 2^{h^2}$ forces $L \geq h^2$); the singleton-context quotient of Lemma 4.4; and, in the proof of Theorem 1.1, $h = \lfloor (n-2)/6 \rfloor \geq 2$ for $n \geq 14$, the floor identity and $8s + 2 \geq 2X \Leftrightarrow s \geq X/4 - 1/4$, and (35) from $8s + 2 \geq 2D(h)$. The write's own code confirmed the floor identity for $14 \leq n \leq 2 \cdot 10^5$ and the exponent coefficient $\log_2(5/2)/54 = 0.0244801499 \dots$ (a truncation). The shipped compiler audit passed again at the write.

5 A separate consequence for binary complementation

The binary compiler also transfers an exponential lower bound when the machine recognizing the complementary language may itself be nondeterministic. This section uses one external algebraic theorem from [8], identified below. The determinization theorem proved earlier is independent of this imported premise.

5.1 The external path-diagram bound

For $m \geq 1$, take disjoint sets D^+, D^- , each with m labels. A path diagram is a quadruple of relations

$$F \subseteq D^+ \times D^+, \quad B \subseteq D^- \times D^-, \quad L \subseteq D^+ \times D^-, \quad R \subseteq D^- \times D^+.$$

The positive sign denotes entry from the left and the negative sign entry from the right. The relations F, B describe through paths; L, R describe returns to the side of entry. Product means joining adjacent boundaries and recording finite paths from an outer entry to the first outer exit. Expanding and contracting traversals of a consecutive block proves associativity. The identity has identity through relations and empty return relations. Write $\mathcal{T}_m$ for this finite monoid and $z \sqsubseteq w$ for inclusion in all four relations. Multiplication preserves $\sqsubseteq$ in both arguments.

Theorem 5.1 (Imported order-reversing image bound). *Let $m \geq 1$, let H be a finite set with $h \geq 2$ elements, and let S_0 be a submonoid of $\mathcal{T}_m$. Suppose a surjective multiplicative map $\phi_0 : S_0 \rightarrow \mathcal{R}_H$ satisfies*

$$z \sqsubseteq w \implies \phi_0(w) \subseteq \phi_0(z) \quad (z, w \in S_0).$$

Then

$$2m \geq 2^{\lfloor (h-2)/127 \rfloor}. \quad (36)$$

Identity preservation of ϕ_0 is not required.

This is the theorem entitled *Order-reversing image bound*, source label `amp:image`, in Section 2 of [8]. Its full hypotheses and numerical conclusion are stated here; its proof by missing recurrent classes and nested corners is not reproduced in this article.

The parameter m counts labels in each direction. Thus the quantity in (36) is the total number $2m$ of directional labels; it is not the degree of the Brauer matching representation used in the determinization proof.

Lemma 5.2 (Finite acceptance and monotonicity). *For an s -state binary 2NFA C in the endmarker model of this article, there are a homomorphism $\tau : \{0, 1\}^* \rightarrow \mathcal{T}_{s+1}$, fixed endmarker diagrams λ, ρ , and positive labels a, b such that*

$$u \in L(C) \iff (a, b) \in F_{\lambda\tau(u)\rho}. \quad (37)$$

Consequently $\tau(u) \sqsubseteq \tau(v)$ implies

$$xuy \in L(C) \implies xvy \in L(C) \quad (x, y \in \{0, 1\}^*). \quad (38)$$

Proof. The finite-computation construction of [8] has the following exact normalization. Add one new success state. From every original accepting state, allow a stay move into it; in the new state sweep right and make one designated artificial exit past the right endmarker. This exit represents acceptance and is not an additional move available to the original machine. A finite computation reaches it precisely when an original finite computation has reached an accepting state, including an accepting initial state.

Use the resulting $s + 1$ states as labels in each direction. For each cell symbol, including markers, a local diagram edge records any finite sequence of stay transitions followed by one exit move from the cell. The entry sign records the side of arrival. Finite computations split into such visits, and local edge witnesses concatenate back into finite computations. This gives (37) with the marker diagrams fixed and the ordinary-cell products defining τ . Missing transitions and infinite nonaccepting computations create no finite successful path. Finally, inclusion of all four relations is preserved by adjoining the diagrams for x, y and the markers. The designated accepting edge therefore persists, proving (38). $\square$

5.2 Encoded contexts give the required antitone quotient

Let $H = [h]$ and let B_h be the binary language constructed above, recognized by a $6h + 2$ -state 1NFA. For a word $w = R_1 \cdots R_t$ over the formal alphabet $\mathcal{R}_H$, write

$$r(w) = R_1 \cdots R_t, \quad \text{enc}(w) = \text{enc}(R_1) \cdots \text{enc}(R_t),$$

and use $r(\varepsilon) = I_H$, $\text{enc}(\varepsilon) = \varepsilon$. The compiler proves

$$\text{enc}(w) \in B_h \iff r(w) \neq \emptyset. \quad (39)$$

The transition tables also define B_h on every other binary word, so complementation below is taken in the full set $\{0, 1\}^*$.

Proposition 5.3 (Direct antitone quotient). *If an s -state 2NFA C recognizes $\{0, 1\}^* \setminus B_h$, a submonoid of $\mathcal{T}_{s+1}$ admits a surjective unital homomorphism onto $\mathcal{R}_H$ that reverses inclusion.*

Proof. Use τ from Lemma 5.2 and let

$$S_0 = \{\tau(\text{enc}(w)) : w \in \mathcal{R}_H^*\}.$$

This is the submonoid generated by the diagrams of the individual encoded relations, including the empty product. Suppose $\tau(\text{enc}(u)) \sqsubseteq \tau(\text{enc}(v))$. Fix $i, j \in H$ and use the binary contexts $x = \text{enc}(I_{\{i\}})$ and $y = \text{enc}(I_{\{j\}})$. For every relation A , the product $I_{\{i\}}AI_{\{j\}}$ is empty exactly when $(i, j) \notin A$. Hence (39), followed by complement acceptance and (38), gives

$$(i, j) \notin r(u) \implies (i, j) \notin r(v).$$

Thus $r(v) \subseteq r(u)$. Equality of the two diagrams gives this inclusion in both directions, so

$$\phi_0(\tau(\text{enc}(w))) = r(w)$$

is well-defined. Concatenation proves multiplicativity, the empty word proves unitality, and every $A \in \mathcal{R}_H$ is the image of $\tau(\text{enc}(A))$. The implication just proved is the required order reversal. $\square$

Theorem 5.4 (Binary nondeterministic complementation). *Under the imported premise of Theorem 5.1, every s -state 2NFA recognizing $\{0, 1\}^* \setminus B_h$ satisfies*

$$2(s+1) \geq 2^{\lfloor (h-2)/127 \rfloor}, \quad s \geq \frac{1}{2} 2^{\lfloor (h-2)/127 \rfloor} - 1. \quad (40)$$

Consequently, for each $n \geq 14$, an n -state binary one-way NFA has a complementary language requiring at least

$$\frac{1}{2} 2^{\lfloor (n-14)/762 \rfloor} - 1 \quad (41)$$

states in every two-way nondeterministic recognizer.

Proof. Apply Theorem 5.1 to Proposition 5.3, with $m = s + 1$. For the last assertion choose $h = \lfloor (n-2)/6 \rfloor$ and add unreachable states to B_h to obtain exactly n states. The identity $\lfloor (h-2)/127 \rfloor = \lfloor (n-14)/762 \rfloor$ gives (41). $\square$

The same homomorphism τ and its $2(s+1)$ directional labels are used throughout this argument. In particular, encoded word length does not introduce a state factor. The external dependency is specifically Theorem 5.1; the encoding and the antitone-quotient argument have been given here. This article provides no new Lean formalization of that imported theorem or of this complementation transfer. The compiler's finite exact audits verify its stated encoding behavior and do not certify the external path-diagram bound.

[write] **Status of this section, 7 October 2026.** Theorem 5.4 is conditional: it holds if Theorem 5.1 does. That premise is a theorem of the companion OpenAI manuscript, not proved here and not checked by the write, which compared only its statement with a copy of the companion's source at the pinned commit (Section 1.5);

it is listed nowhere in this report or its README among the proved results. What is proved here is the transfer: Lemma 5.2 (read; a normalization of the companion’s finite-computation construction, which the write did not read) and Proposition 5.3, whose antitone step $((i, j) \notin r(u) \Rightarrow (i, j) \notin r(v), \text{ hence } r(v) \subseteq r(u))$ the write rederived, as it did the deduction with $m = s + 1$ and the identity $\lfloor (h - 2)/127 \rfloor = \lfloor (n - 14)/762 \rfloor$ (by hand and, for $14 \leq n \leq 2 \cdot 10^5$, by its own code). The determinization theorem, Theorem 1.1, does not use this section.

6 Exact finite audits and reproducibility

The universal claims in this article are proved mathematically. The accompanying programs test independent representations of the objects involved, with special attention to failure modes that are easy to miss in a diagram argument: reversed product conventions, loops and parallel edges, pending binary bits, multiplicity of paths, and spurious dependence on an adjacent input symbol. These computations support the implementation and exposition. They are not finite substitutes for the induction, a proof-assistant formalization, or a certificate of bibliographic novelty.

6.1 Idempotent structure

The script `code/brauer_audit.py` enumerates every perfect matching on $2m$ ports for $0 \leq m \leq 7$, a total of 146,600 diagrams. Its primary test of idempotence computes the literal three-layer graph product e^2 and compares the resulting matching with e . A separate graph predicate tests the alternating-component classification. The enumeration includes nonidempotents, so the classification is not used to generate only purported positive examples.

For the idempotents it checks the sharp support bound, the exact deficit identity, equality structure, fixed-rank maxima, the component-deletion stability statement, and the maximizer count. The observed totals are:

m	All diagrams	Idempotents	Support equality cases
0	1	1	1
1	1	1	1
2	3	2	1
3	15	10	7
4	105	40	25
5	945	296	61
6	10,395	1,936	481
7	135,135	17,872	2,731

An equality case here means $|\text{supp}(e)| = 3 \text{ corank}(e)/2$, including the identity. The counts do not assert that every fixed-rank support maximizer must attain that equality when $m < 3 \text{ corank}(e)/2$.

6.2 The repetition-independent budget

The script `code/idempotent_budget_audit.py` uses literal diagram multiplication from the preceding script. For every ordered idempotent pair through degree four, it tests the relative corner bound along all positive powers up to the first repetition. It also closes each pair of distinct idempotent generators under multiplication and checks every resulting element. The exact coverage is 1,706 ordered corner pairs, 1,898 distinct positive corner powers, and 826 complete generated monoids containing 4,280 inspected elements when counted separately for each generating family.

The same program samples reproducible families in degrees zero through twelve, with the seed recorded in its JSON output. It tests 3,250 corner pairs, 3,250 families of two to four idempotents, 195,000 repeated-word prefixes, and 3,442 final-word powers. Each family also receives a graph check of component invariance, even cap-edge counts, and the identity between the total edge budget and the sum of generator coranks. These larger-degree checks are sampled, and are labeled as such.

6.3 The relation compiler

The script `code/relation_compiler_audit.py` independently constructs the four-letter transition tables and the binary decoder. It compares path counts using arbitrary-precision nonnegative integers, not just Boolean reachability.

Test family	Cases
Every individual relation for $1 \leq h \leq 4$	66,066
Every ordered pair of relations for $1 \leq h \leq 3$	262,404
Singleton entry tests for every relation, $1 \leq h \leq 3$	4,674
Every binary string of length at most twelve, $1 \leq h \leq 4$	32,764
Total	365,908

The arbitrary-string tests include odd lengths and noncanonical even words. They compare the actual pending-bit behavior with direct decoding. For canonical concatenations, the expected path-count matrix is an ordinary integer matrix product, so accidental duplicate paths are detectable.

6.4 Actual symbol-local matching diagrams

The script `code/sparse_slots_audit.py` constructs the local lane graphs themselves, contracts them to perfect matchings, multiplies those matchings, and compares the test-port predicate with direct execution of the original deterministic machine. It does not infer the result by applying a graph-connectivity predicate to an already assembled whole-input graph.

Every legal one-state binary machine in the stated model is tested: there are 288, including both choices of accepting set and all legal partial transition rows at the two ordinary symbols and two markers. An additional 512 two-state machines are sampled with a fixed seed. Each is tested on all 63 binary words of length at most five. The audit constructs 3,200 local diagrams and performs 50,400 machine-word comparisons.

The direct executions include 24,066 initial acceptances, 4,367 later acceptances, 12,637 rejecting infinite runs, and 9,330 undefined rejecting halts. Every local matching is checked to be perfect and to satisfy $k \leq 8s + 2$. This explicitly exercises the corner cases in Section 3.

6.5 Exact recurrence and finite consequences

The script `code/finite_bounds.py` evaluates (21) through $h = 256$, using only integer ceilings. It also verifies the explicit union-graph obstruction in Section 7.3. Selected values are:

h	Source states $6h + 2$	$[F(h)]$	Degree bound $2D(h)$	2DFA states, at least
10	62	2	6	1
20	122	13	18	2
50	302	196	416	52
100	602	19,074	66,452	8,307
200	1,202	1,136,868,378	1,769,699,850	221,212,481

The last column applies (35) to the exact recurrence. The closed form remains useful because it gives a simple exponential statement for all parameters; the recurrence preserves the improvements from parity and the best arm sizes at each finite stage.

6.6 Reproduction and interpretation

From the extracted package directory, run:

```
python3 code/run_all.py --output-dir reproduced
```

All audit programs require only Python’s standard library. The runner invokes them with Python optimization enabled; mathematical checks raise explicit exceptions rather than relying on removable assertion statements. The `certificates` directory contains the executed reference outputs, including the random seeds, parameter ranges, and a case-stream digest for the relation compiler. Timing fields and local output paths may change across runs; mathematical counts and the fixed case-stream digest should agree.

The article can be rebuilt with the supplied Makefile, using `pdflatex` and `bibtex`. TeX source, bibliography, audit programs, reference JSON, exact recurrence CSV, source provenance, theorem ledger, and licensing notices are supplied together. No remote network access is required for compilation or for any mathematical audit.

The independent review performed during preparation checked the algebraic corners, the amplifier, the modular compiler, the machine convention, and the sparse local construction. It was an additional AI-assisted mathematical review, not independent human peer review. The source snapshots and precise theorem dependencies make subsequent human or formal review possible without reconstructing the research session.

[write] **The shipped layout and the reruns, 7 October 2026.** In ProveIt the package’s files are shipped in their delivered layout (the report README lists them); `article.pdf` is the write’s build of this text, and the delivered PDF, `article.bbl` and `SHA256SUMS` are not shipped. The command above writes `reproduced/` inside the directory it is run from, as does `make audit`: in the repository, run it on a copy or with an output directory outside the report. On Windows use `py` for `python3`. The shipped `certificates/relation_compiler_audit.json` records the producer’s scratch output path (`/workspace/scratch/3b2ceb1576c8/...`), a sandbox path with no personal data. At the write, on a copy with Python 3.14.4, the suite passed in 97 s; every certificate equals the shipped one as data except the timing fields, the Python version and that output path, and `finite_bounds.csv` is byte-identical. The counts quoted in this section were also recomputed from their definitions: the 146,600 matchings and both columns of the first table by the write’s own code (for $m \leq 7$), $66,066 = \sum_{h \leq 4} 2^{h^2}$, $262,404 = \sum_{h \leq 3} 4^{h^2}$, $4,674 = \sum_{h \leq 3} h^2 2^{h^2}$, $32,764 = 4(2^{13} - 1)$ and their total 365,908; $288 = 2 \cdot 3 \cdot 3 \cdot 4 \cdot 4$ one-state machines, 63 words, $800 \cdot 63 = 50,400$ comparisons, $800 \cdot 4 = 3,200$ diagrams and the four outcome counts summing to 50,400; the table of finite bounds (all five rows).

7 Further research: precise questions and useful obstructions

The results settle several quantitative issues within the matching-monoid approach, but leave substantial structural problems. The following questions are chosen so that a successful answer would improve a theorem, identify a genuine barrier, or produce a reusable formal component.

7.1 The true degree of a full relation divisor

Let $\beta(h)$ be the least m for which an identity-containing submonoid of $\mathcal{B}_m$ admits a unital surjection onto $\mathcal{R}_{[h]}$. Existence follows from the constructions in this article, and its growth is already pinned

down to exponential order. That order already follows from the earlier exponential source bound and its general machine representation [9]; the following consequence records explicit bounds from the constructions given here.

Proposition 7.1. *For $h \geq 2$,*

$$2(5/2)^{\lfloor (h-2)/9 \rfloor} \leq \beta(h) \leq 12 \cdot 4^h + 26. \quad (42)$$

In particular $\beta(h) = 2^{\Theta(h)}$.

Proof. The lower bound is Theorem 2.13. Consider the four-letter language defined by starting with any state in U and accepting at the end exactly in U , using (27)–(30). A deterministic subset machine has at most 2^{2h} subset states, one separate initial state, and one separate accepting state. On the left marker it enters the subset U ; on each ordinary symbol it takes the relational image of its current subset; on the right marker it enters the accepting state exactly when its subset meets U . All subset states are nonaccepting, so this respects the convention of Section 1.2.

For $a = 4$, Theorem 3.1 gives degree at most $12(2^{2h} + 2) + 2$. Canonical four-letter words and their singleton contexts give the surjection by the same proof as Lemma 4.4, before binary decoding. This is the claimed upper bound. $\square$

Research question 1. What are the exponential rate and leading corrections of $\beta(h)$? Does $\lim_{h \rightarrow \infty} h^{-1} \log_2 \beta(h)$ exist? Can one construct a significantly smaller matching representation of the full relation monoid, or prove a stronger divisor obstruction?

The current proof yields a lower rate $\log_2(5/2)/9$ and an upper rate 2. Establishing a suitable product, submultiplicative, or supermultiplicative relation for β could first settle existence of a rate. A direct representation may improve the upper bound more effectively than determinizing the compiler.

7.2 Beyond rectangular relation addition

Research question 2. Can a nonrectangular family of relation additions improve the gain-to-deletion ratio of Proposition 2.11?

The $p \times q$ grid adds pq independent edges at a cost of $p + q$ conjugates and deletes $p + q - 1$ points when isolating one addition. The choice $(5, 5)$ is optimal only for this architecture. A better construction needs a different incidence pattern, a way to share witnesses without leaving earlier edges in the retained corner, or a stronger simultaneous local lower bound. Optimizing p, q again cannot improve the fixed-parameter rate already proved.

One concrete starting point is a directed acyclic incidence system with several intermediate vertices. Its products must be tracked exactly: accidental length-two paths may add unwanted relations and destroy the singleton-corner property. An optimization problem based only on edge counts would miss this obstruction.

7.3 Minimum generated rank needs more than union connectivity

The budget theorem uses a union graph and parity. Those data need not determine the actual minimum rank in the generated monoid.

Proposition 7.2 (A connected union need not attain its parity floor). *There are two rank-two idempotents in $\mathcal{B}_4$ whose union cap graph is connected, although every nonempty word in them has rank two.*

Proof. Use labels 0, 1, 2, 3. Let a have left cap $\{0, 1\}$, right cap $\{1, 2\}$, and transversals $2 \rightarrow 0$, $3 \rightarrow 3$. Let b have left cap $\{0, 1\}$, right cap $\{1, 3\}$, and transversals $3 \rightarrow 0$, $2 \rightarrow 2$. Each consists of one three-vertex alternating path and one fixed point, so each is idempotent of rank two.

Direct gluing gives $ab = b$ and $ba = a$; the left caps agree, and tracing each of the two remaining transversal paths gives the stated right factor. Thus every nonempty word equals its last letter, and the generated monoid is $\{I, a, b\}$. The union cap graph has edges $\{0, 1\}$, $\{1, 2\}$, and $\{1, 3\}$, with the first edge occurring twice. It is connected on four vertices, for which the component parity lower bound on rank is zero. That value is not attained. $\square$

Research question 3. Which additional data determine the minimum rank of a monoid generated by Brauer idempotents, or characterize equality in the corank budget?

Generator identities, cap colors, and how alternating connectors interact matter beyond uncolored connectivity. A promising formulation is to retain each generator's two colored matchings on each invariant component and study the semigroup of induced endpoint connectors. A useful theorem would characterize the existence of a word attaining the component parity floor, with an effective bound on the length of a witnessing word.

7.4 Optimal compiler size and ambiguity

Research question 4. What is the least asymptotic number of states needed by a fixed-alphabet NFA compiler whose canonical words realize every relation on h designated boundary states?

Our four-letter compiler uses $2h$ relation states and a binary decoder with $6h + 2$ total states in the stated model. The block length is optimal in order by counting, but that argument does not bound compiler states beyond the designated boundary requirement. A smaller binary construction would improve the coefficient in (2) without changing the algebra.

The strong single-block path-count property suggests a related question: can the same state order and block length be retained under a useful bound on ambiguity across concatenations? Single-block uniqueness alone is insufficient; ordinary matrix multiplication already records potentially many intermediate paths. Results for globally unambiguous sources require a new construction or a structural obstruction.

7.5 The constant in sparse matching representations

Research question 5. Can every binary s -state 2DFA be represented in Brauer degree $cs + O(1)$ for a constant $c < 8$, and what is the best universal constant?

Theorem 3.4 rules out sublinear degree, but leaves a gap in the coefficient. The present construction uses two lanes for every distinct crossing triple. Several transitions may share a port scheme if they cannot conflict in any local symbol diagram. Any proposed reuse must preserve target-side locality and distinguish all possible arrivals; looking only at transitions active in one completed input word is not sufficient.

A sharper machine-specific invariant than the number of candidate triples may be a useful first target. For example, one can ask for the minimum number of compatible slot classes under simultaneous local matching constraints, then compare that combinatorial optimum with the number of states.

7.6 Ordered path monoids and nondeterministic complement

Research question 6. Is there a repetition-independent structural budget for idempotents in the ordered path-diagram monoid used in Section 5?

The Brauer proof relies on degree-two colored components and a parity restriction on rank. General path relations branch, so those arguments do not transfer directly. Nevertheless, an invariant controlling the loss of recurrent classes through many repeated idempotents could replace the analogous losses in the imported complementation amplifier. This is a specific route to improving the constant 127, with an explicit reason why the present theorem does not already do so.

7.7 Uniformity and short witnesses

Research question 7. What quantitative length bound can be put on a binary word distinguishing a small candidate 2DFA from B_h , as a function of h and the candidate's number of states?

The encoding of one relation has length $O(h^2)$, but the semigroup proof may take long words and idempotent powers. An exponential state obstruction is therefore not automatically a useful short-input or time lower bound. Bounding the necessary powers and context lengths could illuminate the connection with resource-bounded uniform simulation. Any proposed implication for logarithmic space must explicitly account for constructibility and the relation between witness length and source description size.

7.8 A practical formalization program

Research question 8. Can the new corank budget and sparse representation be formalized as independent reusable modules, then combined with the existing liveness formalization?

The natural first module represents Brauer multiplication by perfect matchings and finite graph connectivity, proves the colored-component classification, and derives the budget. The second module proves the corner reduction and the minimum-rank unit-lifting lemma in a reusable finite-semigroup interface. The third gives the p, q amplification with ambient ranks and relative coranks distinguished in the types. These algebraic modules should be completed before encoding the entire automaton construction.

On the machine side, finite successor graphs and a precise cycle of directed tree edges provide a tractable interface. The local lane graph should be defined once and related both to Brauer multiplication and to the configuration semantics. The existing exact programs can produce regression examples, but a kernel proof must establish the universal correspondence. A final arithmetic layer can prove the decoder counts, floor identities, and the closed recurrence. This order isolates the two most consequential mathematical risks: faithful corner reduction and symbol locality.

7.9 Research priorities

For immediate continuation, the most focused tasks are: formalize the short corank-budget proof; characterize when the generated minimum rank attains the union-component parity bound; and search for a smaller direct binary compiler. Each has a compact mathematical specification and a clear effect on the present results. The longer-term target is the exact exponential behavior of $\beta(h)$, together with a stronger ordered-path invariant for complementation.

[write] **Under Vladimir's standing rule of 4 October 2026, 7 October 2026.** The eight research questions stand as the source states them, and none is answered elsewhere in the repository. The write

found no claim of the source that is unproved as stated outside them, and none that is wrong; nothing was moved here or refuted. Theorem 5.4 is not an unproved claim in this sense, because the source states it as conditional on a named external premise; it stays where it is, marked conditional (Section 5). Proposition 7.1 was rederived ($12(2^{2h} + 2) + 2 = 12 \cdot 4^h + 26$; the subset machine accepts only through its separate accepting state), and Proposition 7.2 was confirmed by the write's own diagram multiplication (a, b idempotent of rank two, $ab = b$, $ba = a$).

References

- [1] Kehinde Adeogun and Christos Kapoutsis. A quadratic lower bound for 2DFAs against one-way liveness. *arXiv:2602.24279v2 [cs.FL]*, 2026. Version 2, revised 6 July 2026.
- [2] Karl Auinger. Krohn–Rhodes complexity of Brauer type semigroups. *Portugaliae Mathematica*, 69(4):341–360, 2012.
- [3] Richard Brauer. On algebras which are connected with the semisimple continuous groups. *Annals of Mathematics*, 38(4):857–872, 1937.
- [4] Igor Dolinka, James East, Athanasios Evangelou, Des FitzGerald, Nicholas Ham, James Hyde, and Nicholas Loughlin. Enumeration of idempotents in diagram semigroups and algebras. *Journal of Combinatorial Theory, Series A*, 131:119–152, 2015. Theorem 5 and Proposition 17; *arXiv:1408.2021v2*.
- [5] James East and Robert D. Gray. Diagram monoids and Graham–Houghton graphs: Idempotents and generating sets of ideals. *Journal of Combinatorial Theory, Series A*, 146:63–128, 2017.
- [6] Christos Kapoutsis. Nondeterminism is essential in small two-way finite automata with few reversals. *Information and Computation*, 222:208–227, 2013.
- [7] Klaus-Jörn Lange, Pierre McKenzie, and Alain Tapp. Reversible space equals deterministic space. *Journal of Computer and System Sciences*, 60(2):354–367, 2000.
- [8] OpenAI. An exponential state lower bound for two-way nondeterministic complementation. <https://github.com/openai/math/tree/adc7f1241b42e322a6451854ab7e4b4c146bf78a/preprints/An-exponential-state-lower-bound-for-two-way-nondeterministic-complementation-September-25-2026>, 2026. Manuscript dated 25 September 2026. Inspected at commit `adc7f1241b42e322a6451854ab7e4b4c146bf78a`, 7 October 2026.
- [9] OpenAI. An exponential two-way deterministic state lower bound for one-way liveness. <https://github.com/openai/math/tree/adc7f1241b42e322a6451854ab7e4b4c146bf78a/preprints/An-exponential-two-way-deterministic-state-lower-bound-for-one-way-liveness-September-25-2026>, 2026. Manuscript dated 25 September 2026. Inspected at commit `adc7f1241b42e322a6451854ab7e4b4c146bf78a`, 7 October 2026.
- [10] OpenAI. Formalization summary: Problem 129. <https://github.com/openai/math/blob/adc7f1241b42e322a6451854ab7e4b4c146bf78a/lean/docs/129.md>, 2026. Repository documentation; scope inspected 7 October 2026.
- [11] Giovanni Pighizzini. Two-way finite automata: Old and recent results. *Electronic Proceedings in Theoretical Computer Science*, 90:3–20, 2012. Historical survey; *arXiv:1208.2755*.
- [12] Jean-Éric Pin. Syntactic semigroups. In Grzegorz Rozenberg and Arto Salomaa, editors, *Handbook of Formal Languages*, volume 1, pages 679–746. Springer, Berlin–Heidelberg, 1997.
- [13] ProveIt contributors. Aligned fragments and constrained crossover. <https://github.com/VladimirReshetnikov/ProveIt/tree/670a67d35eda2fc92f2483b1b3d643bad9173113/SetTheory/Cardinals/docs/reports/automata-and-formal-languages/constrained-crossover-closure>, 2026. Seven-part research report, 29 September–1 October 2026. README inspected at commit `670a67d35eda2fc92f2483b1b3d643bad9173113`.

- [14] ProveIt contributors. A cyclic-coloring obstruction to maximal binary DFAO reversal. <https://github.com/VladimirReshetnikov/ProveIt/tree/670a67d35eda2fc92f2483b1b3d643bad9173113/SetTheory/Cardinals/docs/reports/automata-and-formal-languages/dfao-reversal-coloring-obstruction>, 2026. Six-part research report, 20–30 September 2026. README inspected at commit 670a67d35eda2fc92f2483b1b3d643bad9173113.
- [15] ProveIt contributors. The six-state shuffle bound. <https://github.com/VladimirReshetnikov/ProveIt/tree/670a67d35eda2fc92f2483b1b3d643bad9173113/SetTheory/Cardinals/docs/reports/automata-and-formal-languages/shuffle-six-state-bound>, 2026. Research report prepared for Vladimir Reshetnikov, 20 September 2026. README inspected at commit 670a67d35eda2fc92f2483b1b3d643bad9173113.
- [16] William J. Sakoda and Michael Sipser. Nondeterminism and the size of two way finite automata. In *Proceedings of the Tenth Annual ACM Symposium on Theory of Computing*, STOC '78, pages 275–286, New York, NY, USA, May 1978. Association for Computing Machinery.
- [17] Michael Sipser. Halting space-bounded computations. *Theoretical Computer Science*, 10(3):335–338, 1980.
- [18] TheoremDB research record R816. The one-way-liveness bound transfers to a fixed binary alphabet. <https://www.theoremdb.org/records/R816/>, 2026. Original reduction reported as written and audited 28 July 2026; self-reported evidence; accessed 7 October 2026. Cite Adeogun and Kapoutsis separately for the imported quadratic bound. CC0-1.0.
- [19] Qiqi Yan. Lower bounds for complementation of ω -automata via the full automata technique. arXiv:0802.1226, 2008.